



Kernel Bypass



Mainack Mondal
Sandip Chakraborty

CS 60203
Autumn 2026

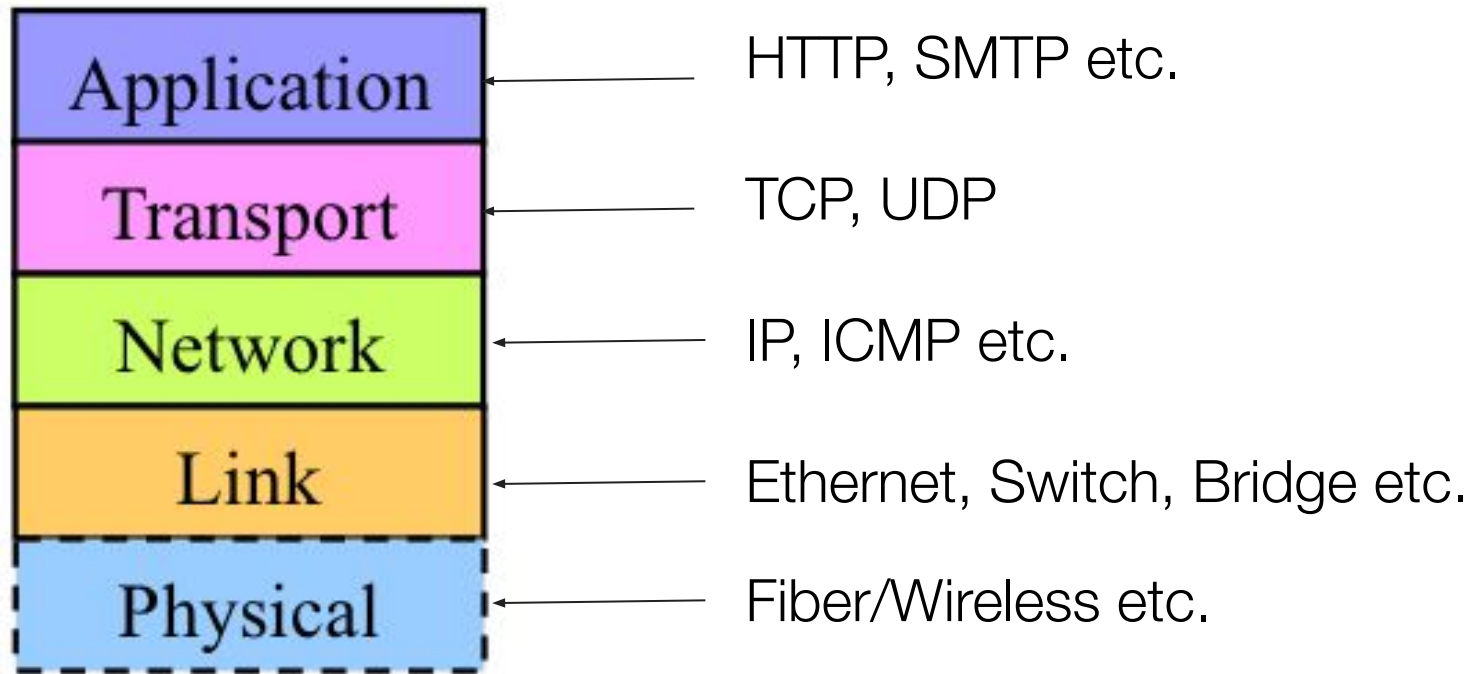
Outline

- Linux Network Stack
 - Receive path, NAPI, transmit path
- Need for Kernel Bypass
 - Per-packet time budget and kernel overheads
- Kernel Bypass Techniques
 - Core ideas: direct NIC access, polling, batching
 - User-space packet processing: DPDK and Netmap
 - In-kernel fast path: XDP and AF_XDP; RDMA
 - User-space network stack: mTCP
- Challenges and when to use kernel bypass

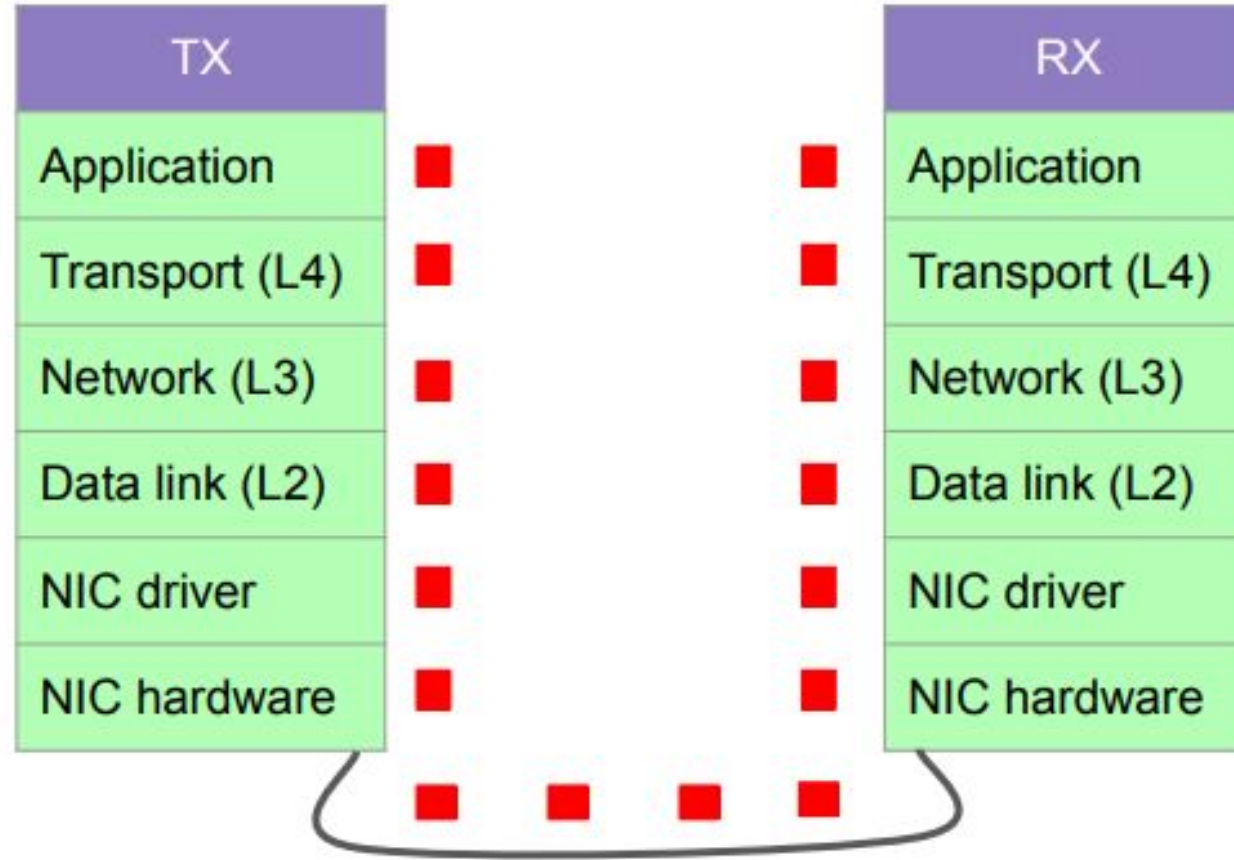
Linux Network Stack

(Slides credits: Mythili Vutukuru, IIT Bombay)

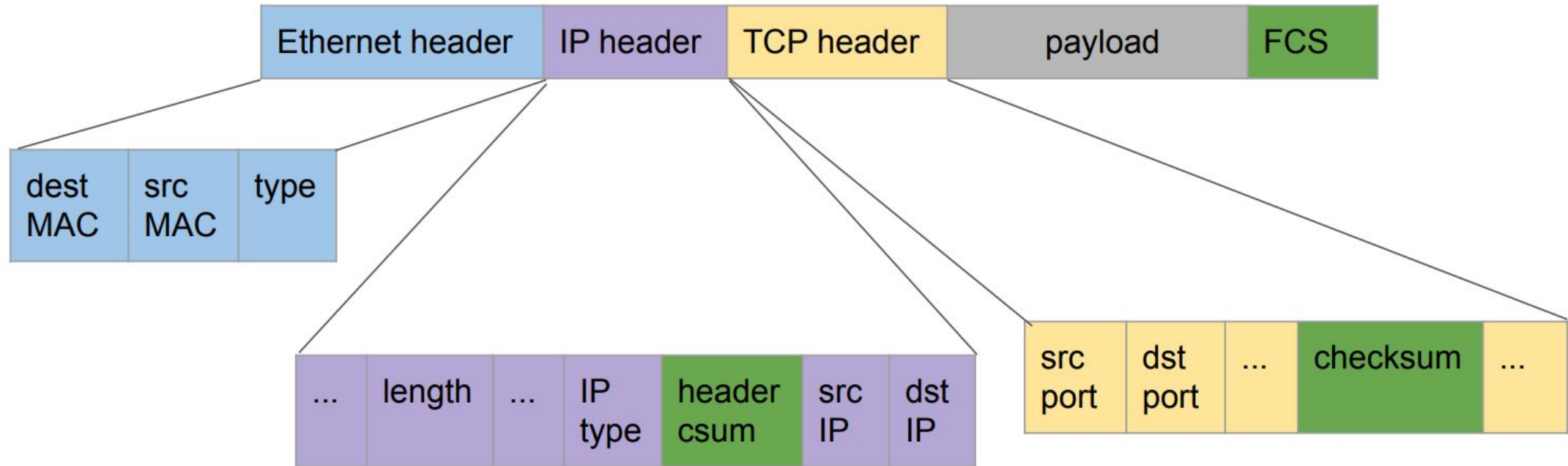
TCP/IP Layers



Typical Packet Flow

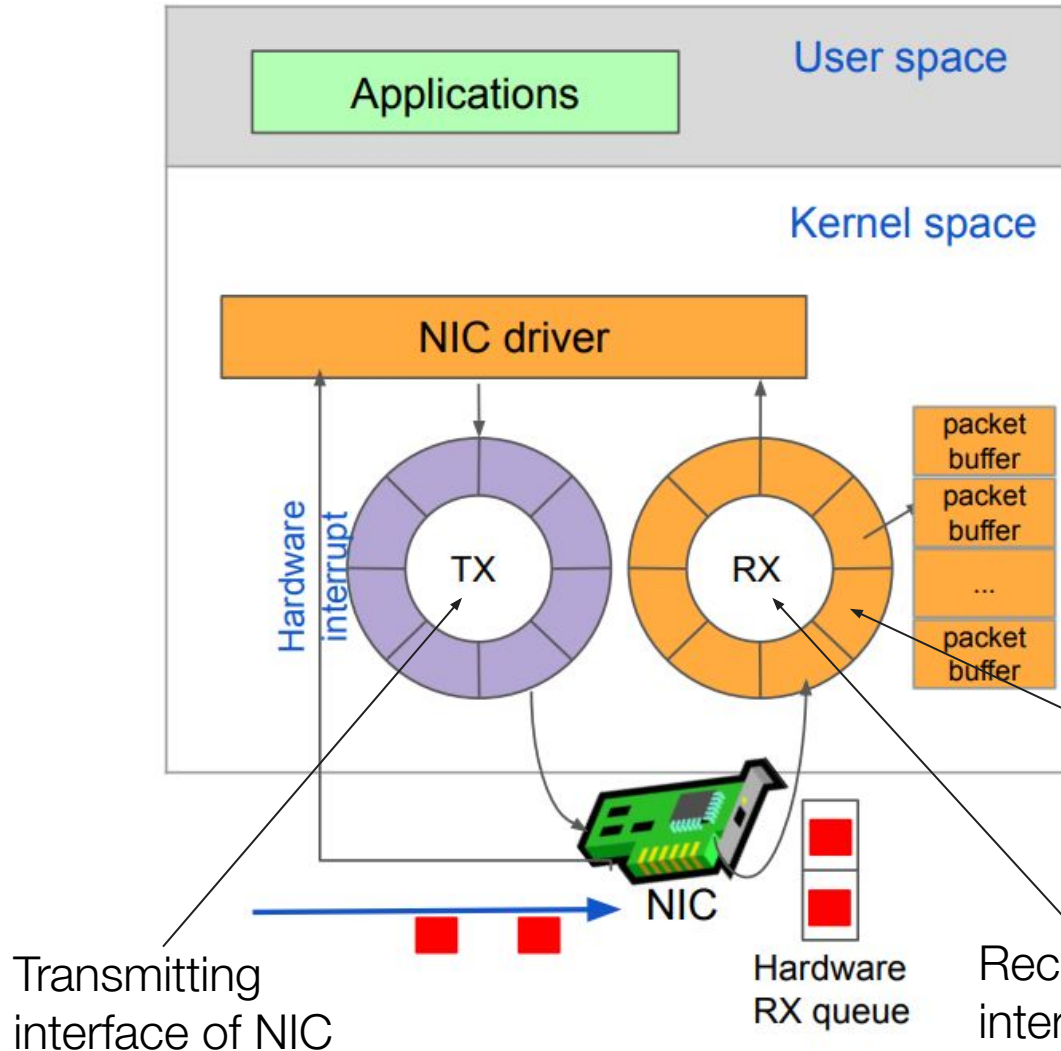


Packet Contents



Let's see the journey of a packet through the
Linux network stack

Packet Arrives at NIC



NIC receives the packet

- Match destination MAC address
- Verify Ethernet checksum (FCS)

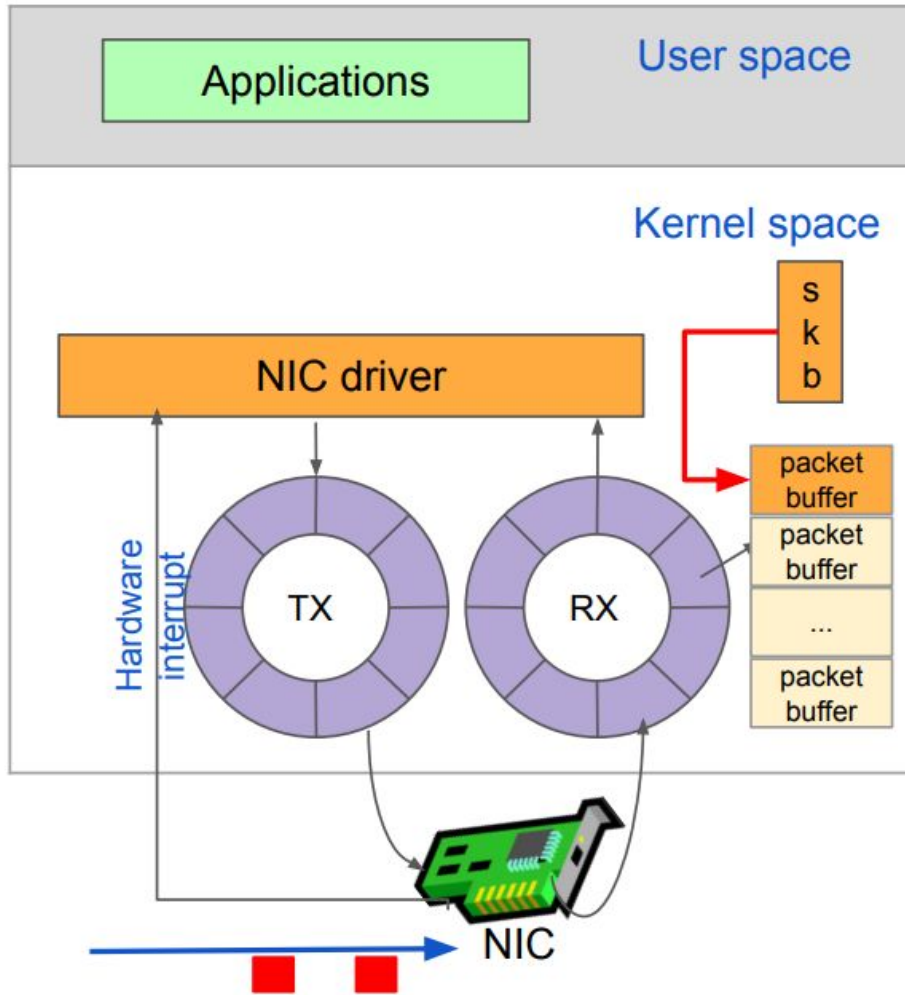
Packets accepted at the NIC

- DMA the packet to RX ring buffer
- NIC triggers an interrupt

TX/RX rings

- Circular queue
- Shared between NIC and NIC driver
- Content: Length + packet buffer pointer

Processing the Packet in Kernel



Driver dynamically allocates an `sk_buff(skb)`

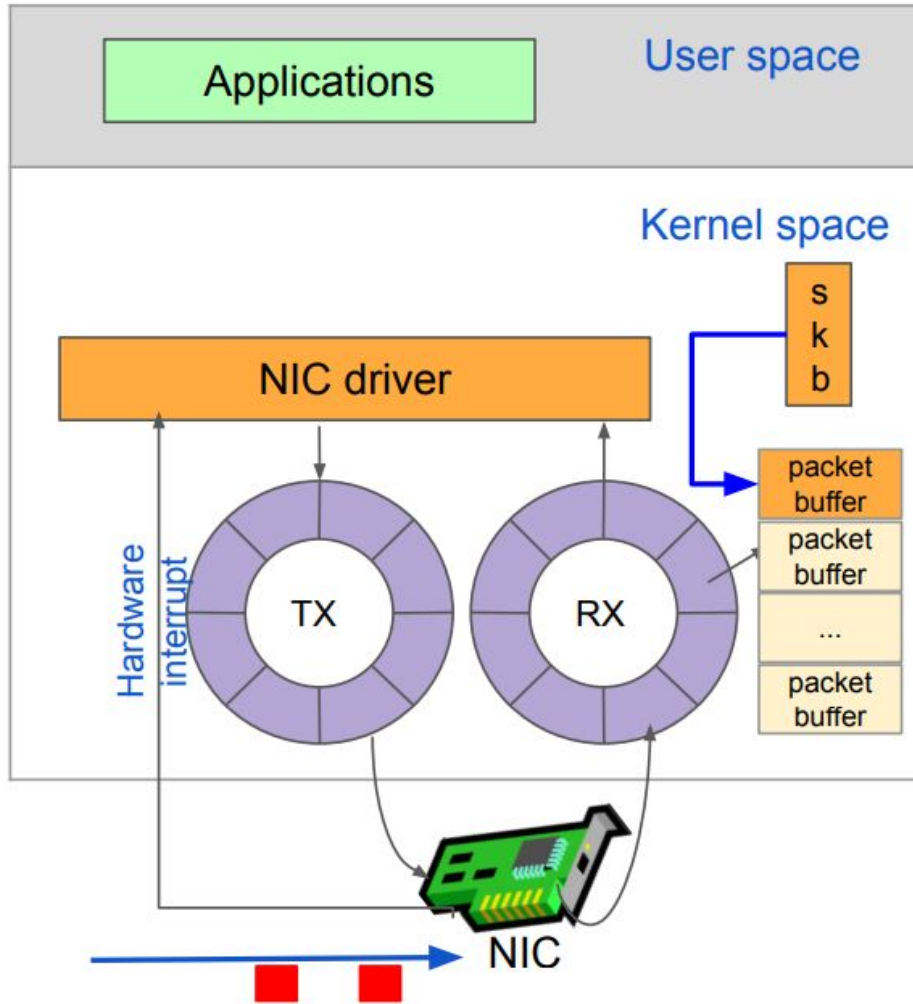
`sk_buff`

In-memory data structure that contains packet metadata

- Pointers to packet headers and payload
- More packet related information ...

To know more about `sk_buff`: Read [Link](#)

Packet Processing (Contd.)

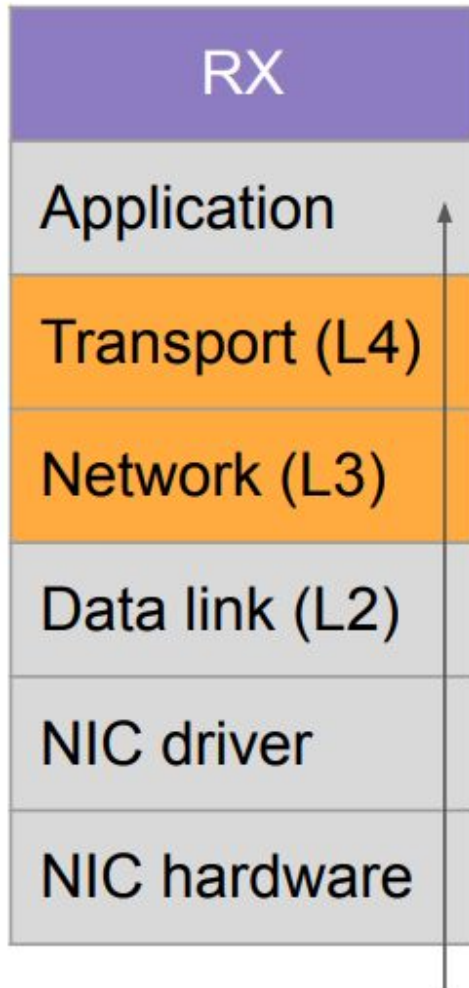


NIC driver processing

- Driver dynamically allocates an **sk-buff**
- Update **sk-buff** with packet metadata
- Remove the Ethernet header
- Pass **sk-buff** to the network stack
(for all packets in buffer)

Call L3 protocol handler

L3/L4 Processing



Common Processing

- Match destination IP/socket
- Verify checksum
- Remove header

+

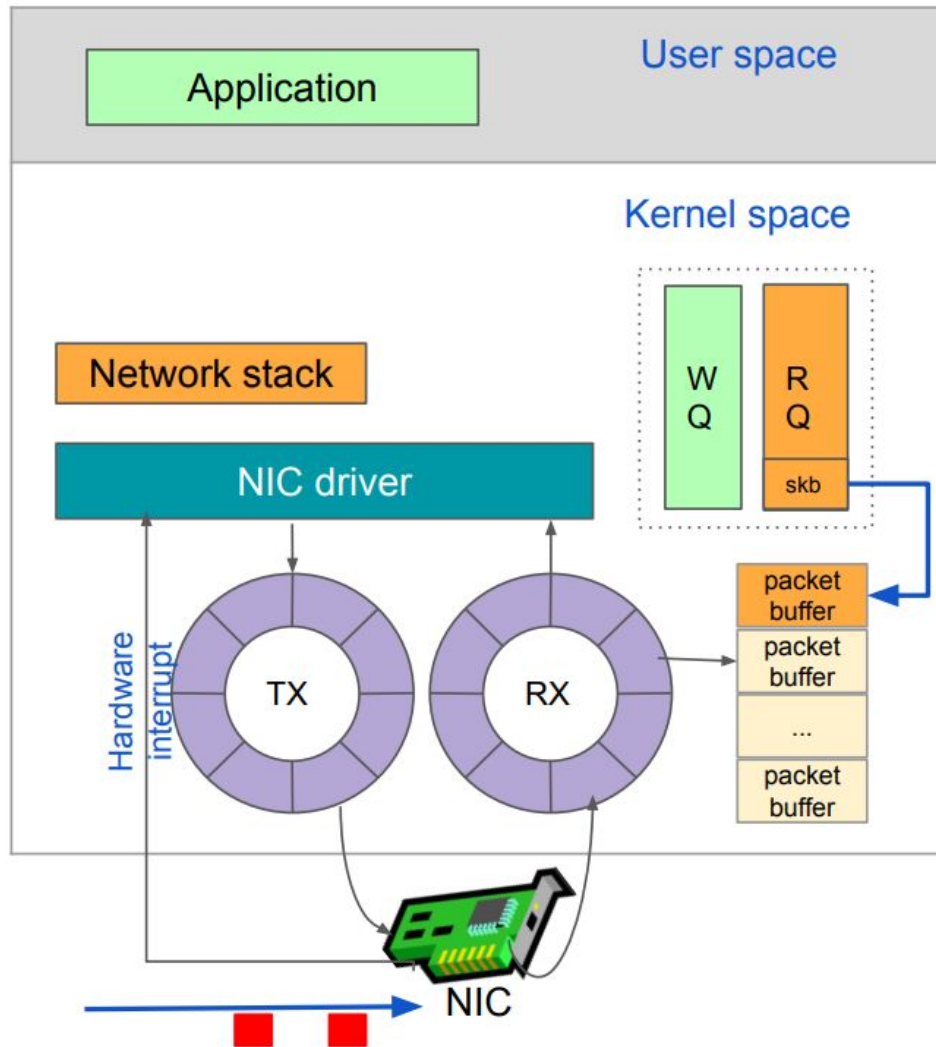
L3-specific processing

- Route lookup
- Combine fragmented packets
- Call L4 protocol handler

L4-specific processing

- Handle TCP state machine
- Enqueue to socket read queue
- Signal the socket

L3/L4 Processing (Contd.)



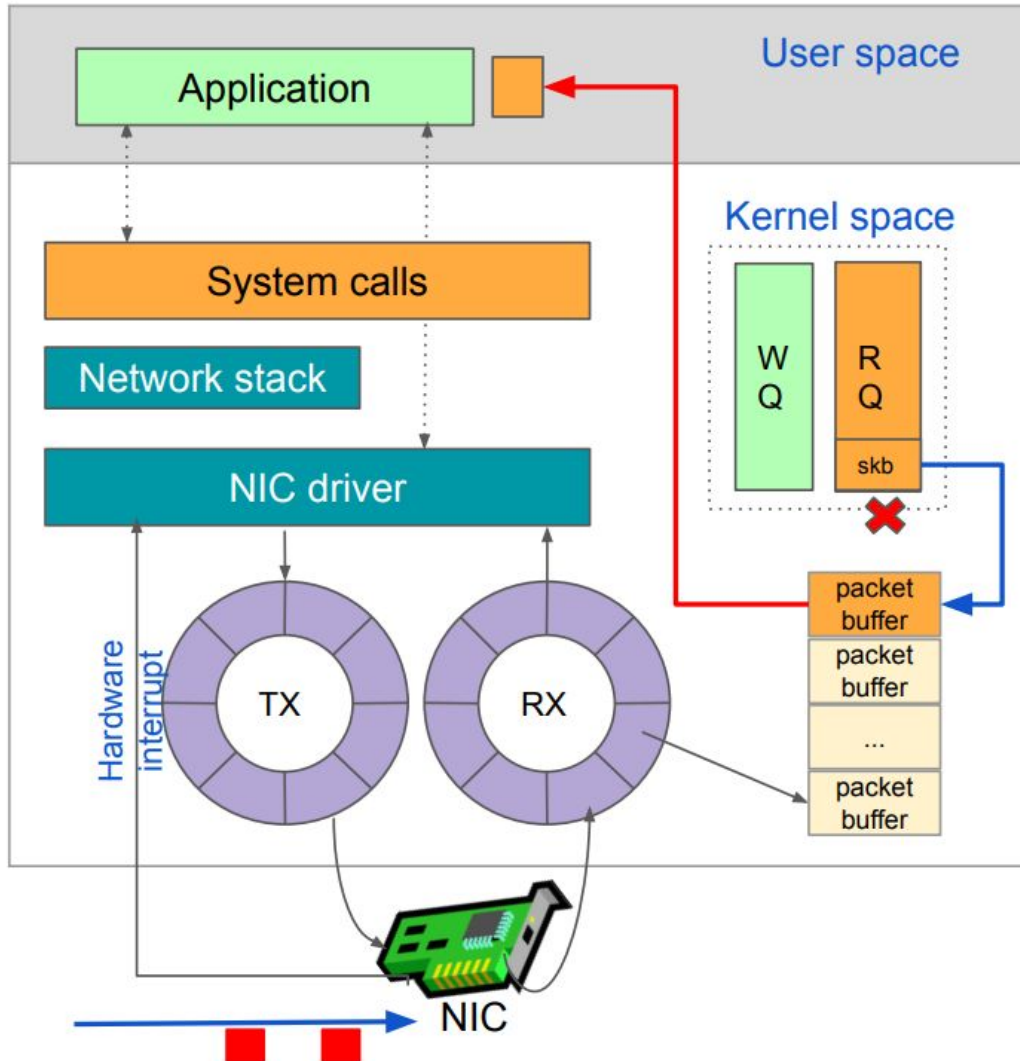
L3-specific processing

- Route lookup
- Combine fragmented packets
- Call L4 protocol handler

L4-specific processing

- Handle TCP state machine
- Enqueue to socket read queue
- Signal the socket

Application Layer Processing



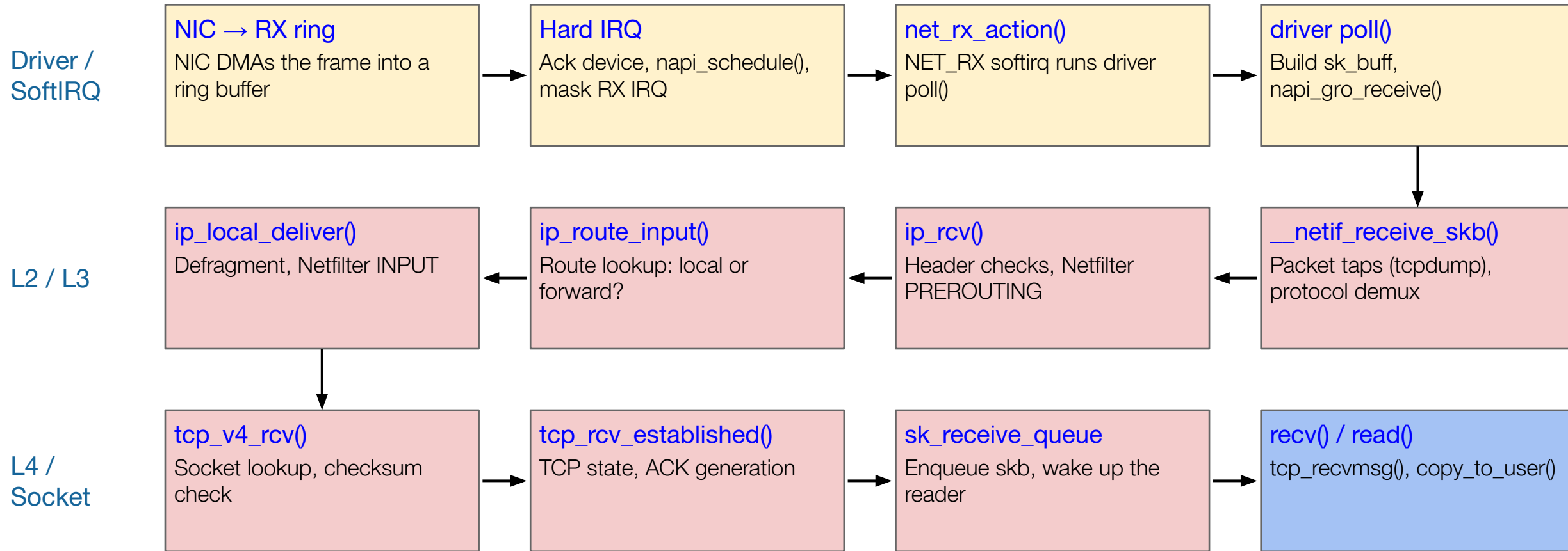
On socket read:

user space to kernel space

- Dequeue packet from socket receive queue (kernel space)
- Copy packet to application buffer (user space)
- Release `sk-buff`
- Return back to the application

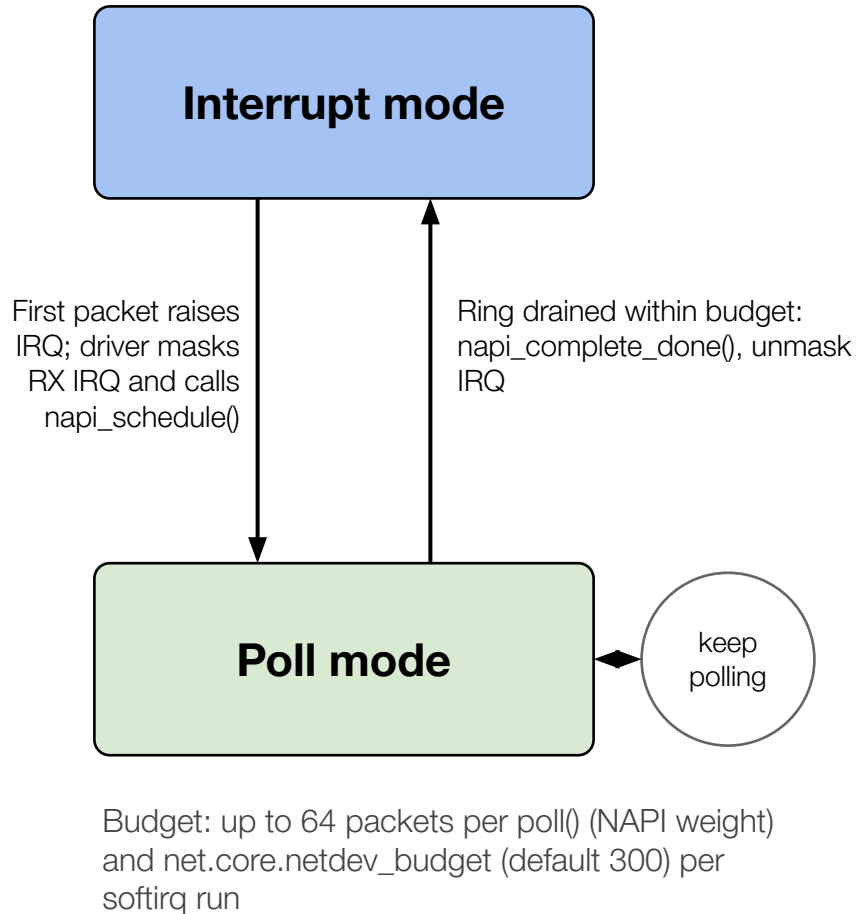
kernel space to user space

Receive Path in One Picture: Linux Call Chain



Every packet walks this whole path, touching many shared kernel structures. The application sees the data only at the very last step.

NAPI: Linux Already Mixes Interrupts and Polling



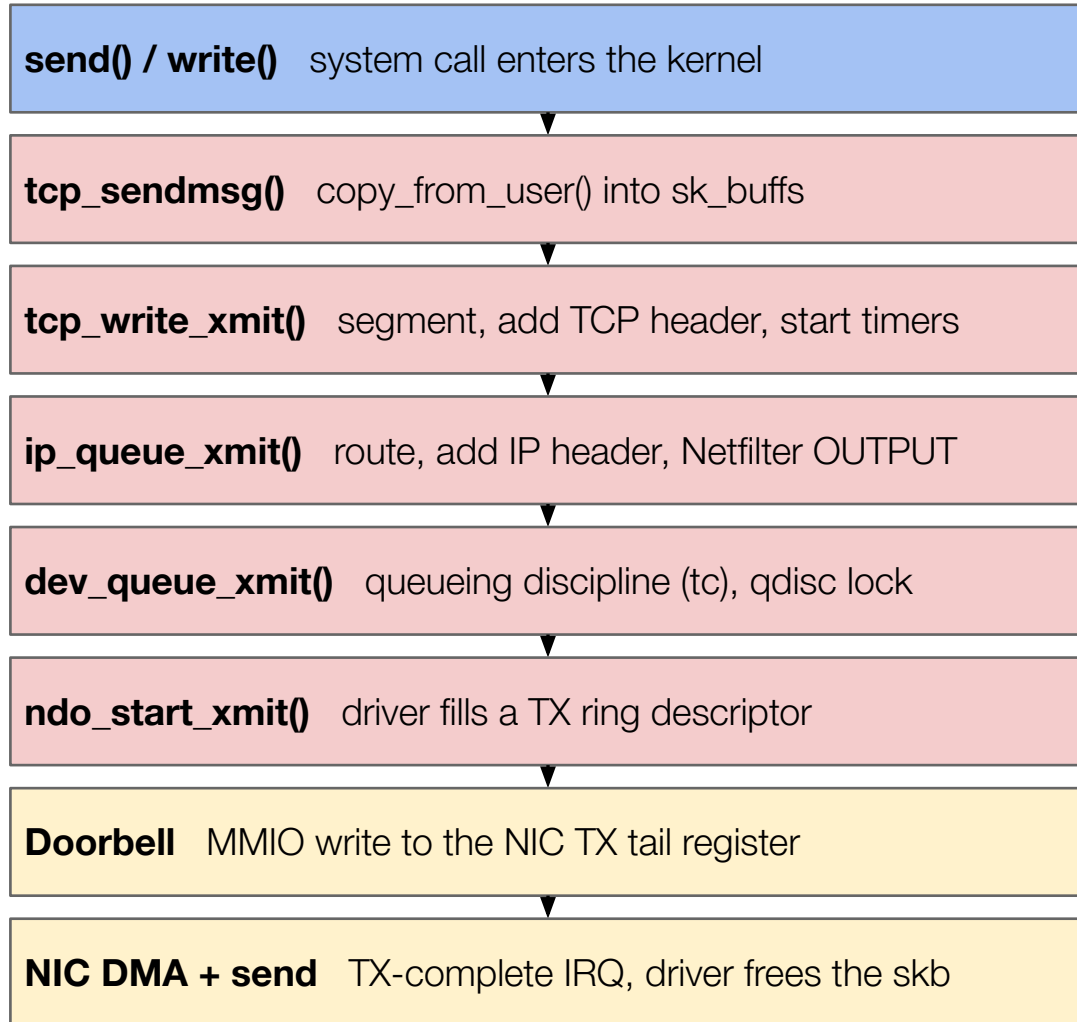
What NAPI fixes

- No interrupt per packet under load: one IRQ, then a batch of packets is polled
- Avoids **receive livelock**: the CPU is not stuck only servicing interrupts
- Low load still gets interrupt-driven, energy-efficient behaviour

What NAPI does not fix

- An sk_buff is still allocated and freed for every packet
- Payload is still copied between kernel and user buffers
- Application still needs a syscall (and often a wake-up) to get data
- Full protocol path and shared socket state remain

The Transmit Path



Costs on the TX side

- **Copy** of every byte from user memory into kernel buffers
- **Locks:** socket lock, qdisc lock, TX queue lock
- **Doorbell per packet:** an MMIO write is an uncached PCIe transaction (hundreds of ns). Linux batches with xmit_more, but only when the stack has more packets ready
- **TX completion:** another interrupt and another free of the sk_buff
- Offloads help bulk traffic: TSO/GSO hand one large buffer to the NIC, which cuts it into MTU-size segments

Small packets get little help from these offloads

And the process goes on ...

Need for Kernel Bypass

How Much Time Do We Have per Packet?

Worst case = minimum-size frames. 64 B frame + 8 B preamble/SFD + 12 B inter-frame gap = **84 B = 672 bits** on the wire per packet.

Link speed	Max packet rate	Time per packet	Cycles @ 3 GHz
1 Gbps	1.49 Mpps	672 ns	~2,000
10 Gbps	14.88 Mpps	67.2 ns	~200
25 Gbps	37.2 Mpps	26.9 ns	~80
40 Gbps	59.5 Mpps	16.8 ns	~50
100 Gbps	148.8 Mpps	6.7 ns	~20
400 Gbps	595.2 Mpps	1.7 ns	~5

67.2 ns

per packet at 10 Gbps

One DRAM cache miss costs about 70 to 100 ns. It alone can blow the 10 Gbps budget.

- Big packets are easier: a 1500 B frame at 10 Gbps leaves ~1.2 μ s. But small packets are common (TCP ACKs, DNS, VoIP, RPCs, key-value requests), and attacks such as SYN floods use minimum-size frames.

What Do Common Operations Cost?

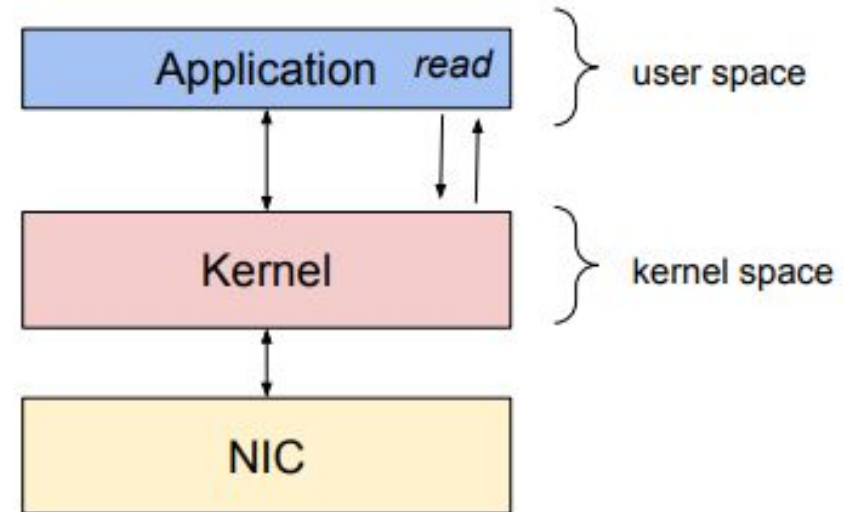
Operation	Approximate cost	Why it matters for packets
L1 cache hit	~1 ns (4 cycles)	Fine. Data already near the core
Last-level (L3) cache hit	~10 to 20 ns	A few per packet already eat the 10G budget
DRAM access (cache miss)	~70 to 100 ns	sk_buff, socket, descriptor misses
Contended atomic / lock	~50 to 200+ ns	Cache line bounces between cores
System call entry + exit	~0.1 to 1+ μ s	Higher with Spectre/Meltdown mitigations
Interrupt + context switch	~1 to 5 μ s	Also pollutes caches and TLB
Copy 1500 B (in cache)	~50 to 100 ns	Per packet, kernel to user

Order-of-magnitude values; they vary with CPU, kernel version and configuration.

Rizzo (netmap, USENIX ATC 2012) measured close to **1 μ s per packet** for a single sendto() on FreeBSD. That caps one core near **1 Mpps**, far below the 14.88 Mpps of 10 GbE.

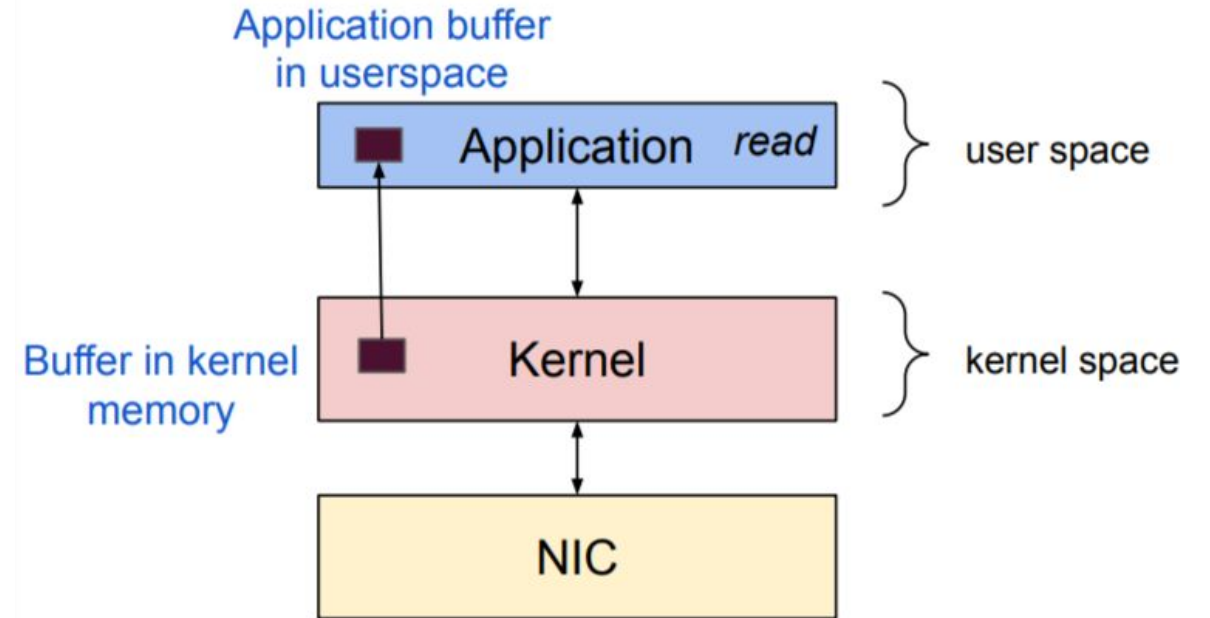
Packet Processing Overheads in Kernel

Context switch between kernel and userspace



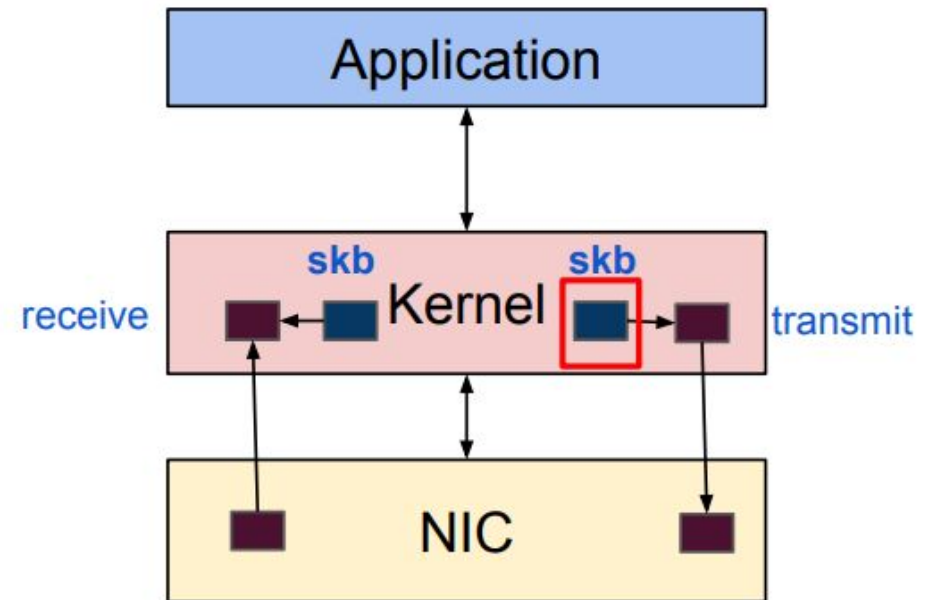
Packet Processing Overheads in Kernel

Packet copy between kernel and userspace

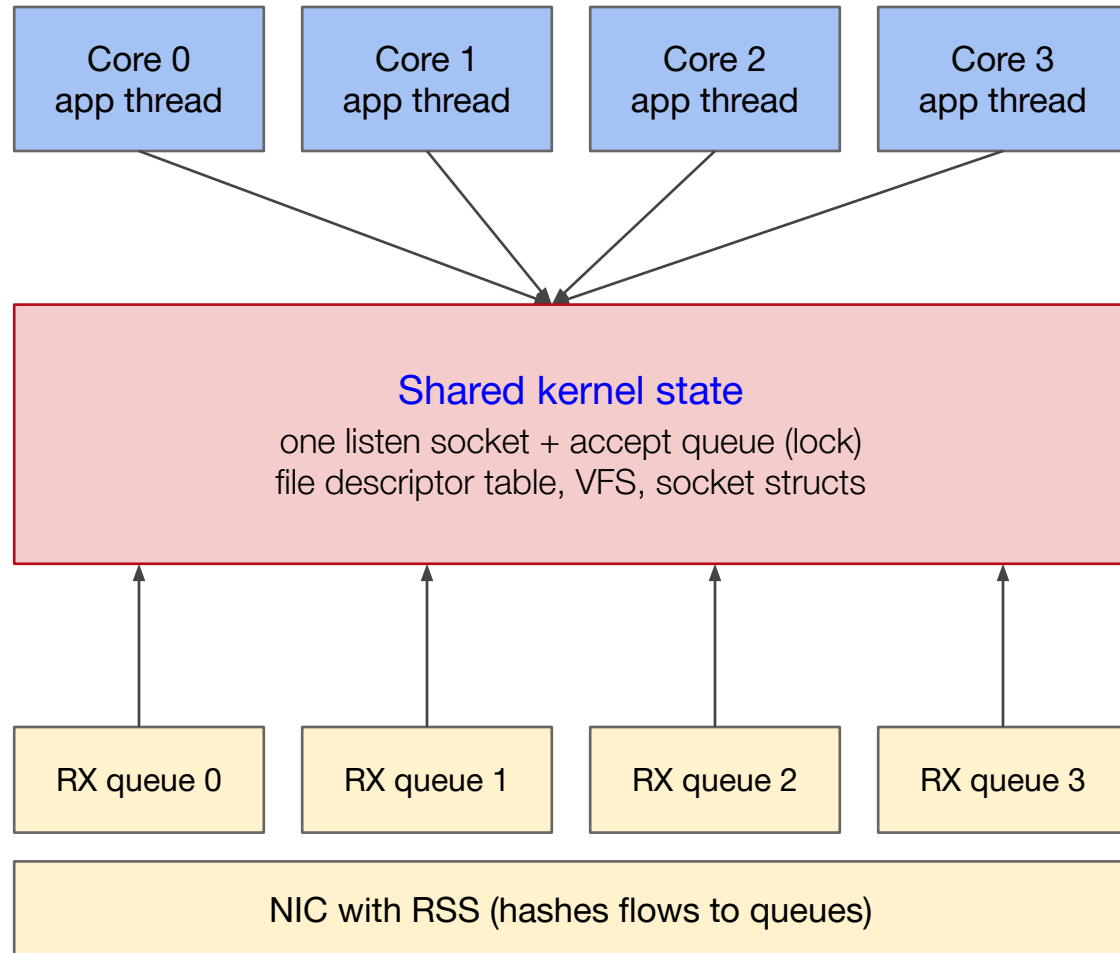


Packet Processing Overheads in Kernel

- Dynamic allocation of sk_buff
- Per packet interrupt
- Shared data structures



Why It Gets Worse on Multicore



- **Shared listen socket:** all cores fight for one accept queue lock (before `SO_REUSEPORT`)
- **No connection locality:** RSS delivers a flow's packets on core A, but the thread reading the socket runs on core B. Data and socket state cross cores
- **Sockets are files:** allocating descriptors and VFS bookkeeping use shared, locked structures
- **Cache line bouncing** of shared counters and socket fields
- Result: for short connections, throughput stops growing as cores are added (shown by Affinity-Accept, MegaPipe and mTCP papers)

Fixes Inside the Kernel, and Why They Are Not Enough

Technique	What it helps	What still remains
Interrupt coalescing, NAPI	Fewer interrupts under load	Adds latency; per-packet stack cost stays
RSS / RPS / RFS	Spread packets over cores, better locality	Shared socket structures, full stack per packet
GRO/LRO, TSO/GSO	Fewer, larger skbs through the stack	Helps bulk TCP, not small packets or many flows
SO_REUSEPORT	One listen socket per core	Still syscalls, copies, sk_buffs
sendfile, MSG_ZEROCOPY	No copy on TX for large data	Page pinning cost; receive side still copies
Busy polling (SO_BUSY_POLL)	Lower latency, no wait for IRQ	Still the whole kernel stack
io_uring	Batches syscalls, async I/O	Kernel still does protocol work per packet

Each fix removes one cost. **Kernel bypass** takes the kernel off the data path altogether.

Summary

- Too many context switches!! → Pollutes CPU cache
- Per-packet interrupt overhead
- Queuing delays
- Dynamic allocation of sk-buff
- Packet copy between kernel and user space
- Shared data structures
- Too Bad!! in multicore

Cannot achieve line-rate for recent high speed NICs!! (40Gbps/100Gbps)

Kernel Bypass to the Rescue



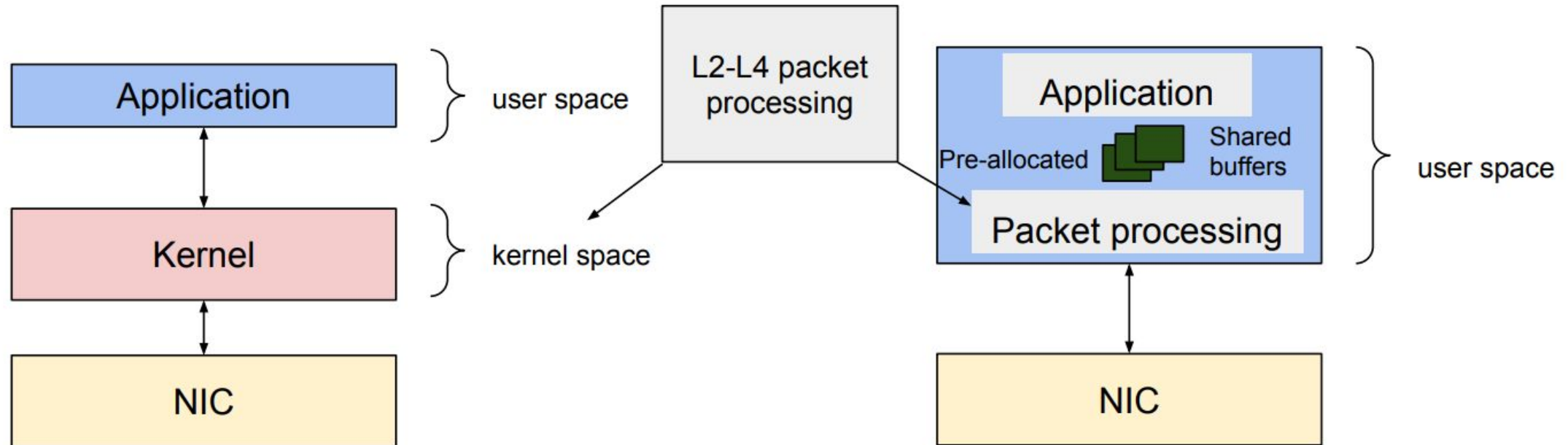
Image credits: Matt Brown

Outline

- Linux Network Stack
 - Receive path, NAPI, transmit path
- Need for Kernel Bypass
 - Per-packet time budget and kernel overheads
- **Kernel Bypass Techniques**
 - Core ideas: direct NIC access, polling, batching
 - User-space packet processing: DPDK and Netmap
 - In-kernel fast path: XDP and AF_XDP; RDMA
 - User-space network stack: mTCP
- Challenges and when to use kernel bypass

Kernel Bypass Techniques

Overcome Overheads in Kernel: Kernel Bypass



- **No** Context switch between kernel and userspace
- **No** Packet copy between kernel and userspace
- **No** Dynamic allocation of `sk_buff`

Yay !!

Core Ideas Behind Kernel Bypass

1. Direct NIC access

- Map NIC registers and descriptor rings into the process
- Driver runs as a library inside the app
- Kernel only does setup (UIO / VFIO)

2. Poll, do not wait

- A dedicated core spins on the RX ring
- No interrupt, no context switch, no wake-up
- Costs a full core even when idle

3. Batch everything

- Receive and send bursts of 32 to 64 packets
- One doorbell or syscall per burst
- Amortises fixed costs, keeps caches warm

4. Zero-copy buffers

- Buffers allocated once, at start-up
- NIC DMAs straight into app-visible memory
- No per-packet malloc/free, no copies

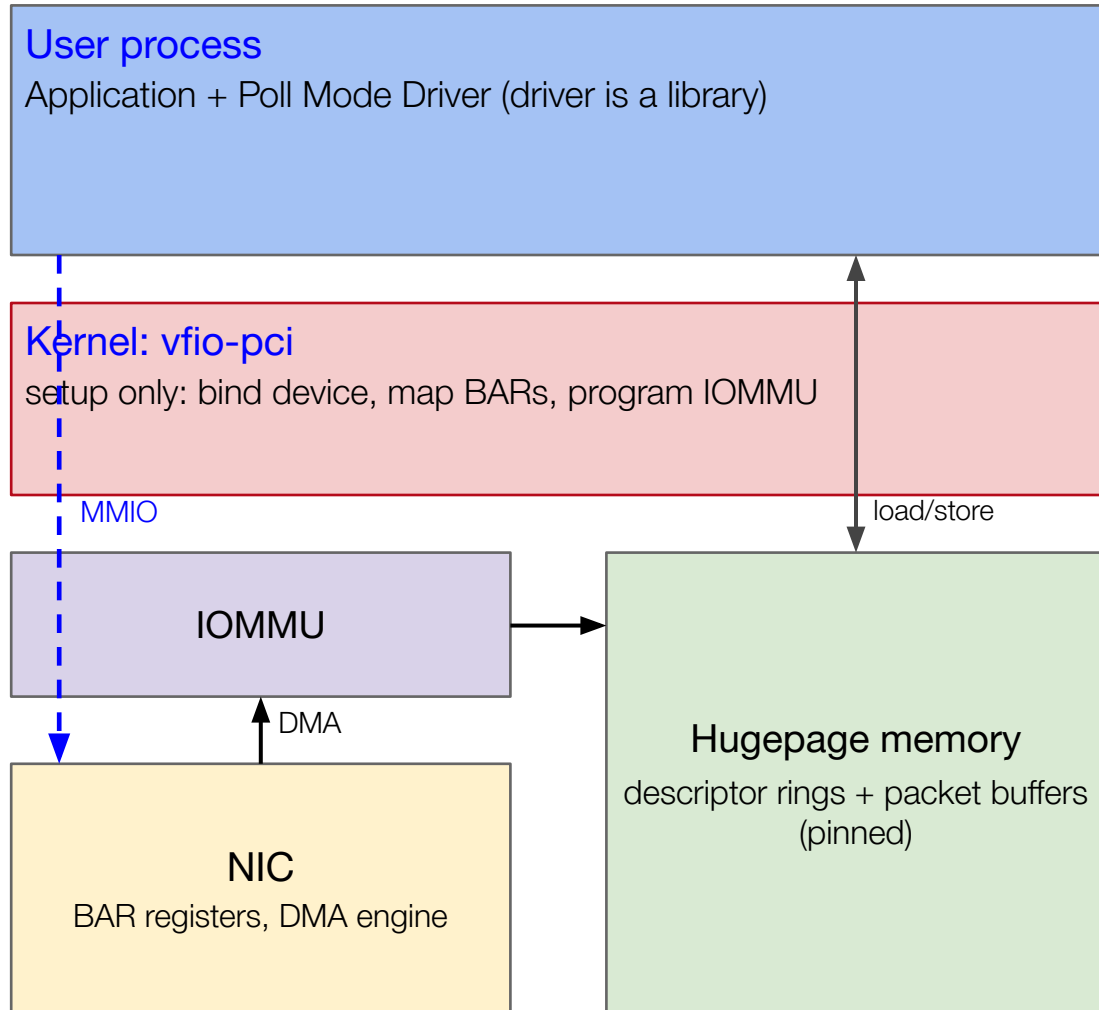
5. Cache and TLB friendly

- Hugepages (2 MB / 1 GB) cut TLB misses
- Cache-line aligned structures, prefetch
- NUMA-local memory for each NIC and core

6. Share nothing

- One pinned thread per core
- One NIC queue per core via RSS
- Lock-free rings, per-core state and caches

How Can User Space Touch the NIC?



- **1. Unbind** the NIC from its kernel driver; bind it to vfio-pci (or igb_uio, uio_pci_generic)
- **2. mmap() the PCI BARs**: the process reads and writes NIC registers directly (doorbells, ring pointers)
- **3. Allocate pinned hugepage memory** and give its I/O addresses (IOVA) to the NIC for DMA
- **4. IOMMU** restricts DMA to memory the process owns, so a buggy app cannot overwrite the kernel
- The interface disappears from the kernel (ip link no longer shows it)

```
$ dpdk-devbind.py --status
$ dpdk-devbind.py --bind=vfio-pci 0000:03:00.0
```

But how does your userspace programs know
when a packet has arrived?

Interrupt vs Poll Mode: Kernel Bypass Techniques

Interrupt Mode



- NIC notifies it needs servicing
- Interrupt is a hardware mechanism
- Handled using interrupt handler
- Interrupt overhead for high speed traffic

Netmap

Poll Mode



- CPU keeps checking the NIC
- Polling is done with help of control bits(**Command-ready** bit)
- Handled by the CPU
- Consumes CPU cycles but handles high speed traffic

DPDK

Interrupts vs Polling: The Trade-off

Aspect	Interrupt-driven	Polling
CPU at low load	Mostly idle, energy efficient	100% busy, wastes energy
CPU at high load	Interrupt storm, risk of livelock	Stable; all cycles go to packets
Latency	IRQ + wake-up + context switch (μ s)	Packet seen almost at once (sub- μ s)
Jitter	Depends on scheduler and IRQ load	Very predictable on a pinned core
Used by	Linux default, NAPI as a hybrid	DPDK PMD, netmap busy wait, SPDK

Receive livelock

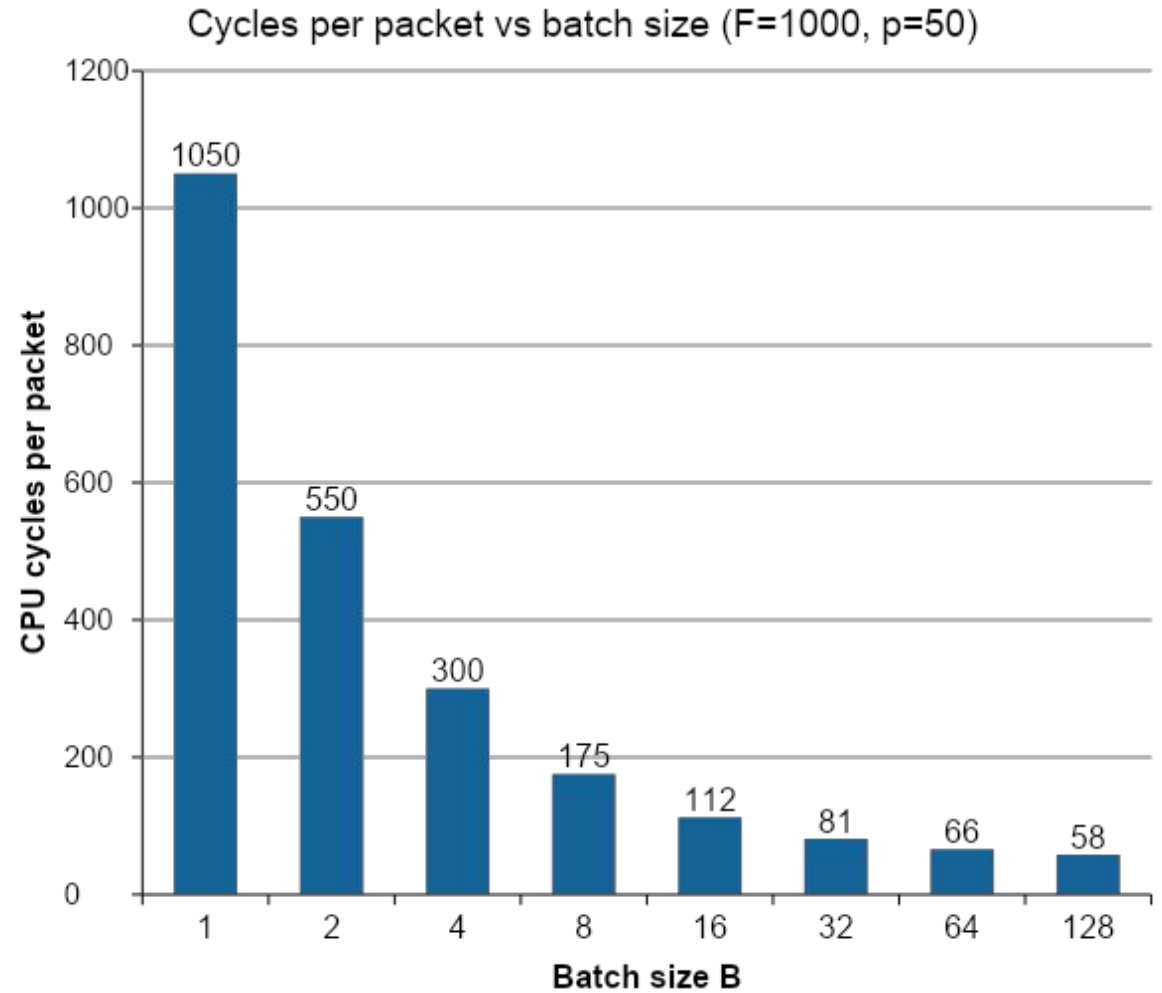
Mogul and Ramakrishnan (1997): under overload, the CPU spends all its time taking interrupts and queuing packets, and no packet is ever fully processed.

Hybrids

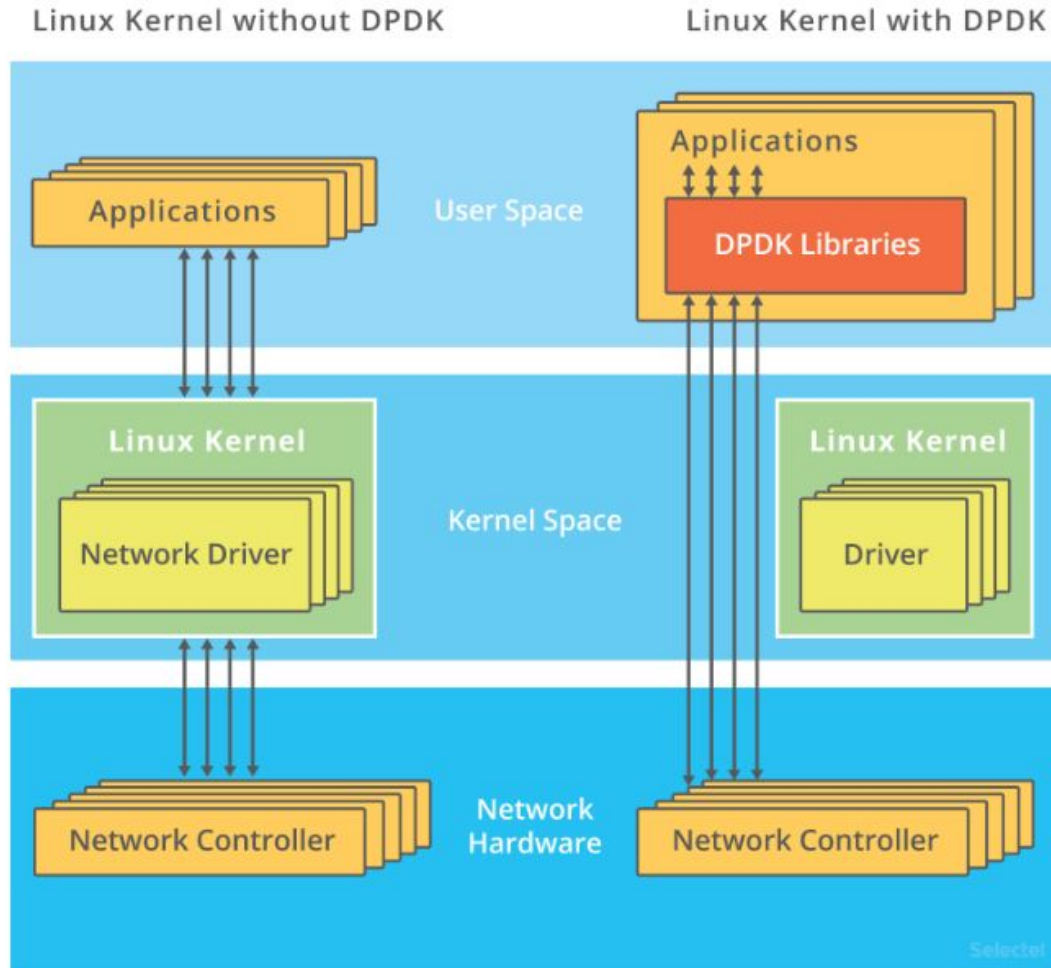
NAPI in Linux; DPDK Rx interrupt mode with power management (l3fwd-power example); adaptive polling and batching in IX.

Batching: Amortise the Fixed Costs

- Every I/O step has a **fixed cost F** (syscall, doorbell MMIO write, function call, cache warm-up) and a **per-packet cost p**.
- With batch size B:
$$\text{cost per packet} = F / B + p$$
- Example on the right: $F = 1000$ cycles, $p = 50$ cycles. Going from $B = 1$ to $B = 32$ cuts the cost per packet about 13 times.
- **Trade-off**: waiting to fill a batch adds latency. DPDK's `rx_burst()` returns whatever is ready (up to B), so it never waits.



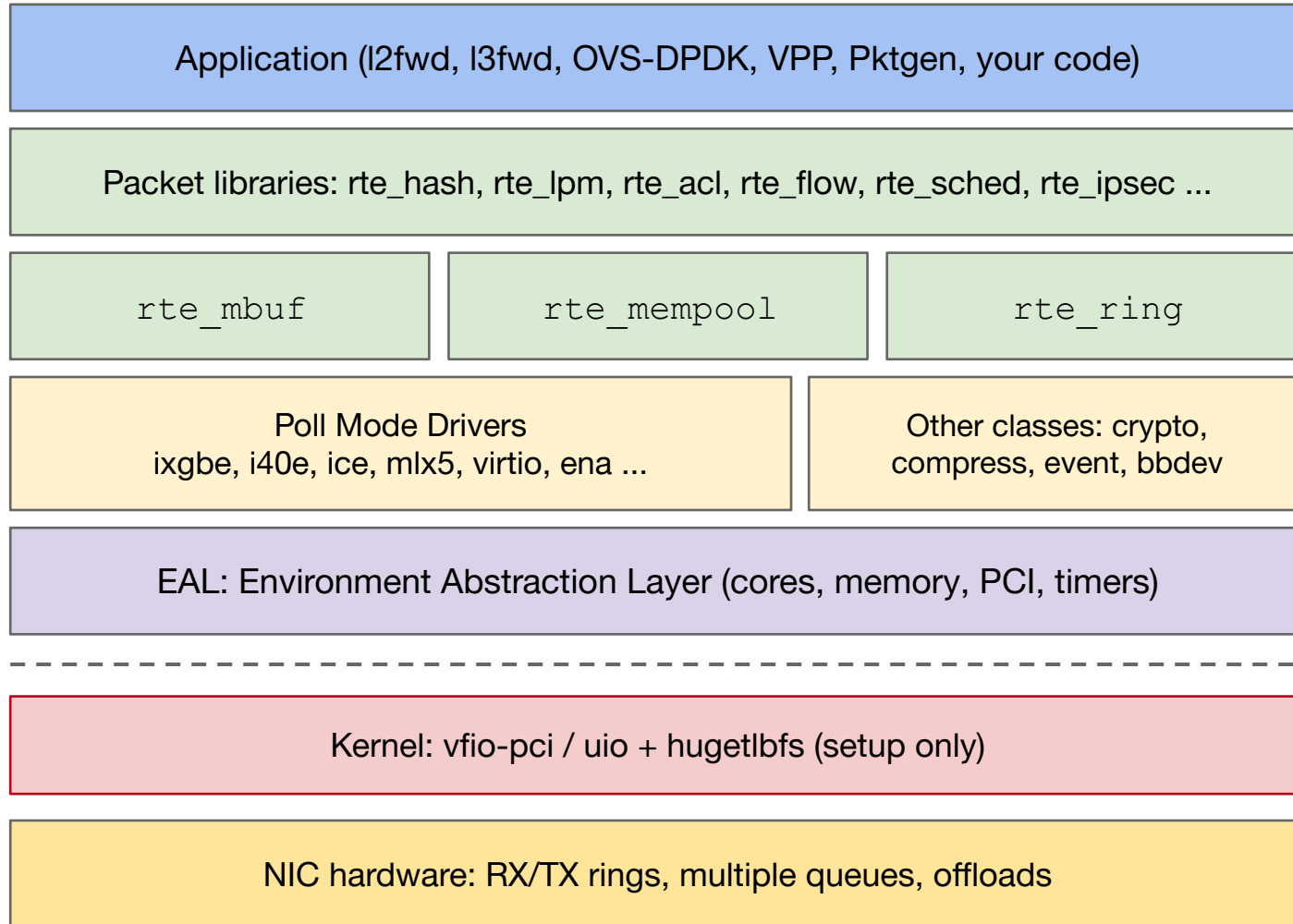
DPDK: Dataplane Development Kit



- **User-space packet processing**
 - Avoid context switching overhead
- **Poll Mode Driver (PMD)**
 - Avoid interrupt processing overhead
 - Keeps a core busy
- **Memory usage optimizations**
 - Light-weight mbufs
 - Memory pools that use hugepages, cache alignment, etc
 - Lockless ring buffers

Source: <https://blog.selectel.com/introduction-dpdk-architecture-principles/>

DPDK Architecture: The Building Blocks



- **EAL** hides OS details: hugepage memory, core pinning, PCI probing
- **PMDs** are user-space drivers: they read and write descriptors directly
- **mbuf / mempool / ring** are the core data structures for packets
- **Libraries** give fast lookups, classification, QoS, crypto
- Kernel is involved only at start-up
- Open source (BSD), Linux Foundation project since 2017

EAL: Environment Abstraction Layer

- First call of every DPDK program: **rte_eal_init()**
- Reserves hugepage memory per NUMA socket
- Scans the PCI bus, attaches a PMD to each device bound to vfio/uio
- Creates one thread per **lcore**, pinned to a CPU with affinity
- Launches work on lcores with `rte_eal_remote_launch()`
- Gives timers, logging, atomics, cycle counter (`rte_rdtsc()`)
- Command-line options:
 - `-l 0-3` cores to use
 - `-n 4` memory channels
 - `-a 0000:03:00.0` allow this NIC

```
int main(int argc, char **argv)
{
    int ret = rte_eal_init(argc, argv);
    if (ret < 0)
        rte_exit(EXIT_FAILURE, "EAL init failed\n");
    argc -= ret; argv += ret;    /* app args */

    unsigned lcore;
    RTE_LCORE_FOREACH_WORKER(lcore)
        rte_eal_remote_launch(worker_main,
                               NULL, lcore);

    worker_main(NULL);    /* main lcore too */
    rte_eal_mp_wait_lcore();
    return 0;
}
```

```
$ sudo ./app -l 0-3 -n 4 -a 0000:03:00.0 -- -p 0x1
```

Hugepages: Why DPDK Needs Them

4 KB

standard page: 64 TLB entries cover 256 KB

2 MB

huge page: 64 TLB entries cover 128 MB

1 GB

gigantic page: 64 TLB entries cover 64 GB

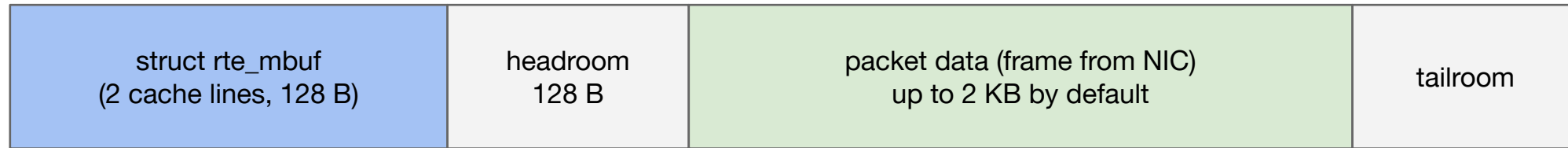
- A packet pool of a few hundred MB touched at random: with 4 KB pages almost every access misses the TLB and needs a page walk
- Hugepages are **pinned**: never swapped or moved, so the I/O address given to the NIC stays valid
- Large **physically contiguous** chunks make DMA simple
- Reserve at boot (best for 1 GB) or at run time; DPDK maps them through hugetlbfs

```
# 1024 x 2 MB pages at run time
echo 1024 > /sys/kernel/mm/hugepages/\
    hugepages-2048kB/nr_hugepages
mkdir -p /mnt/huge
mount -t hugetlbfs nodev /mnt/huge

# 1 GB pages: kernel boot line
default_hugepagesz=1G hugepagesz=1G
hugepages=8
```

The 64-entry figure is an illustration; real TLB sizes differ per CPU and page size.

rte_mbuf: The Light-weight Packet Buffer



one object, allocated once from a mempool in hugepage memory, reused forever

`buf_addr`

`buf_addr + data_off`

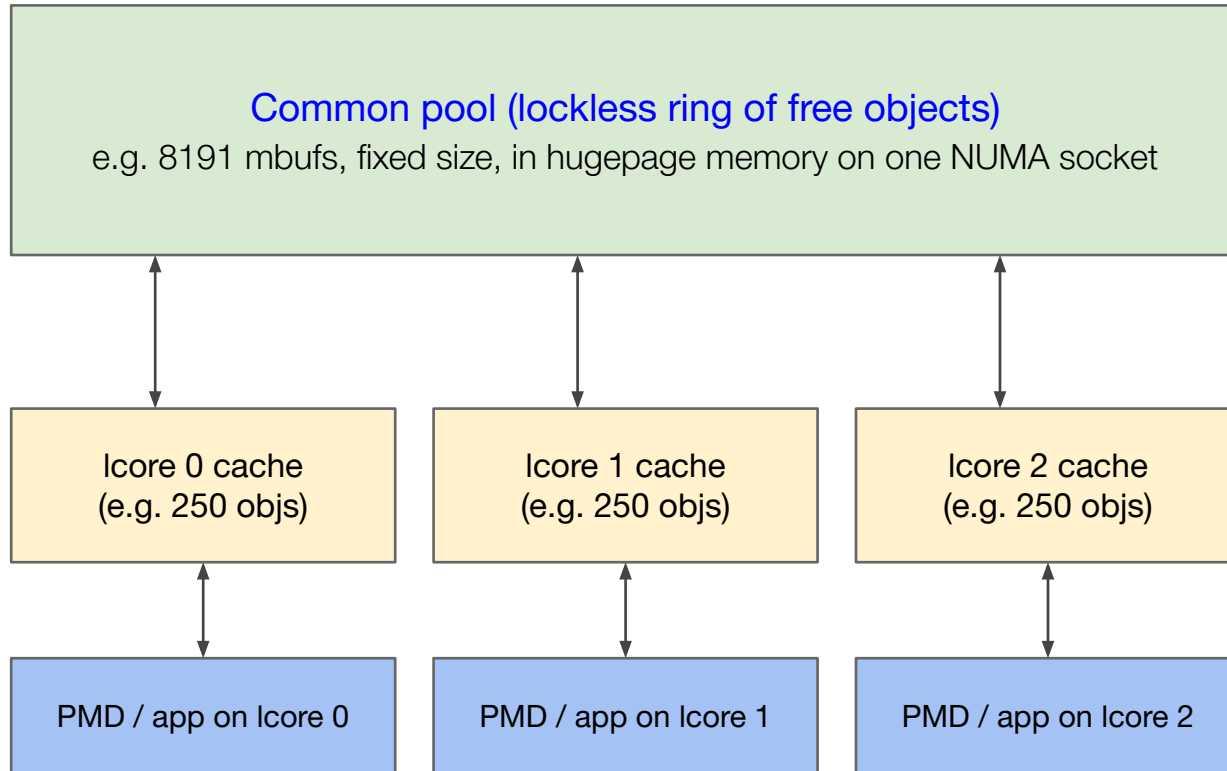
Key fields

- `pkt_len`, `data_len`: total and per-segment length
- `next`, `nb_segs`: chain segments for jumbo frames
- `port`, `packet_type`, `hash.rss`: filled by the PMD
- `ol_flags`: offload requests and results (checksum, VLAN)
- `refcnt`, `pool`: reuse and return to the right pool

Compared with `sk_buff`

- `sk_buff`: metadata struct (over 200 B) from the slab per packet; data allocated separately
- `mbuf`: metadata and data in one fixed object, preallocated
- Headroom lets you prepend headers (VLAN, tunnels) without copying

rte_mempool: Preallocated, Per-core Cached



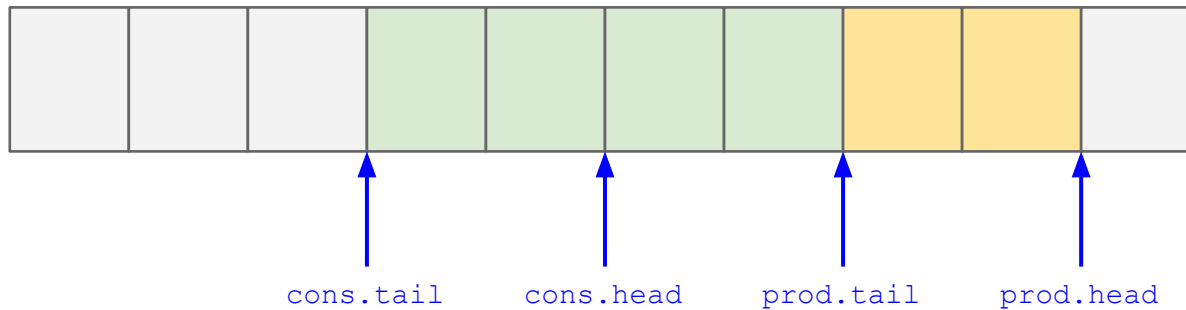
Bulk refill / flush only when a cache is empty or full

- All buffers created at start-up; nothing is allocated on the fast path
- Most get/put calls touch only the **local per-core cache**: no atomics, no locks
- Bulk get/put of many objects in one call
- Objects cache-aligned and padded to spread across memory channels

```
struct rte_mempool *mp =
    rte_pktmbuf_pool_create("MBUF_POOL",
        8191, 250, 0,
        RTE_MBUF_DEFAULT_BUF_SIZE,
        rte_socket_id());
m = rte_pktmbuf_alloc(mp); /* ... */
rte_pktmbuf_free(m);
```

rte_ring: A Lockless FIFO

ring of object pointers (size = power of 2)



green: queued objects yellow: slots reserved by a producer, being written

- Fixed size, stores pointers (e.g. to mbufs)
- **Two-step update**: move head with compare-and-swap to reserve slots, copy objects, then move tail to publish
- Single-producer / single-consumer modes skip CAS entirely
- **Burst** enqueue/dequeue: one atomic for many objects
- Used as the mempool backend and to pass packets between cores

```
struct rte_ring *r = rte_ring_create("rx2wk",
    1024, rte_socket_id(),
    RING_F_SP_ENQ | RING_F_SC_DEQ);
rte_ring_enqueue_burst(r, (void **)bufs, n, NULL);
rte_ring_dequeue_burst(r, (void **)bufs, 32, NULL);
```

Poll Mode Driver: The Forwarding Loop

Setup (once)

- `rte_eth_dev_configure()`: number of RX/TX queues, offloads
- `rte_eth_rx_queue_setup()`: ring size, mempool for buffers
- `rte_eth_tx_queue_setup()`
- `rte_eth_dev_start()`

Fast path (forever)

- `rx_burst` → process in place → `tx_burst`
- No syscall, no interrupt, no copy

```
#define BURST_SIZE 32

static void lcore_main(uint16_t port)
{
    struct rte_mbuf *bufs[BURST_SIZE];

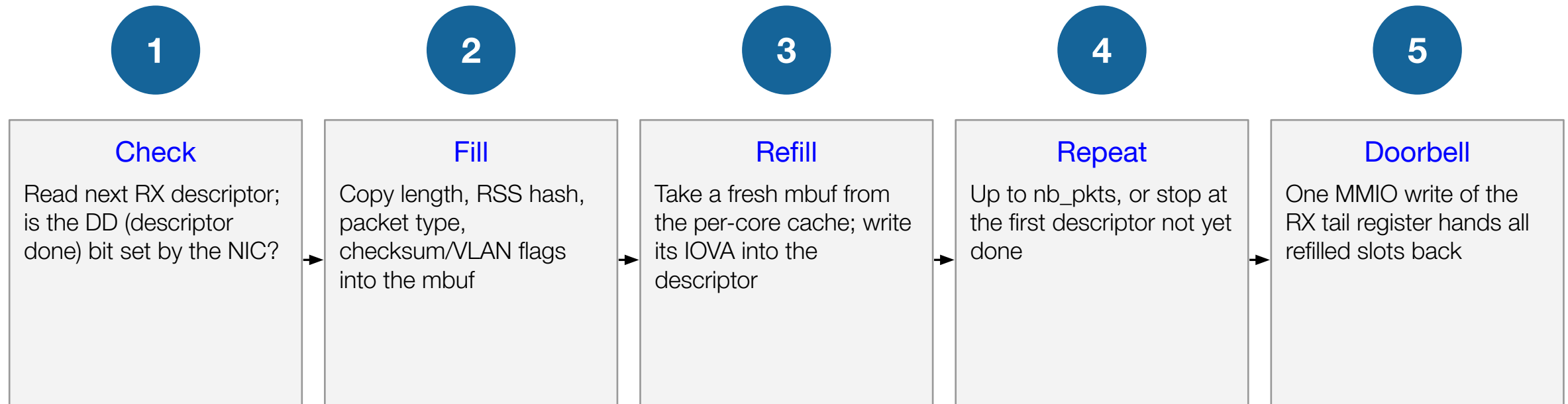
    for (;;) {
        /* busy poll */
        uint16_t nb_rx = rte_eth_rx_burst(
            port, 0, bufs, BURST_SIZE);
        if (nb_rx == 0)
            continue;

        for (uint16_t i = 0; i < nb_rx; i++)
            process(bufs[i]); /* edit in place */

        uint16_t nb_tx = rte_eth_tx_burst(
            port ^ 1, 0, bufs, nb_rx);

        for (uint16_t i = nb_tx; i < nb_rx; i++)
            rte_pktmbuf_free(bufs[i]);
    }
}
```

Inside rte_eth_rx_burst()



Why it is fast

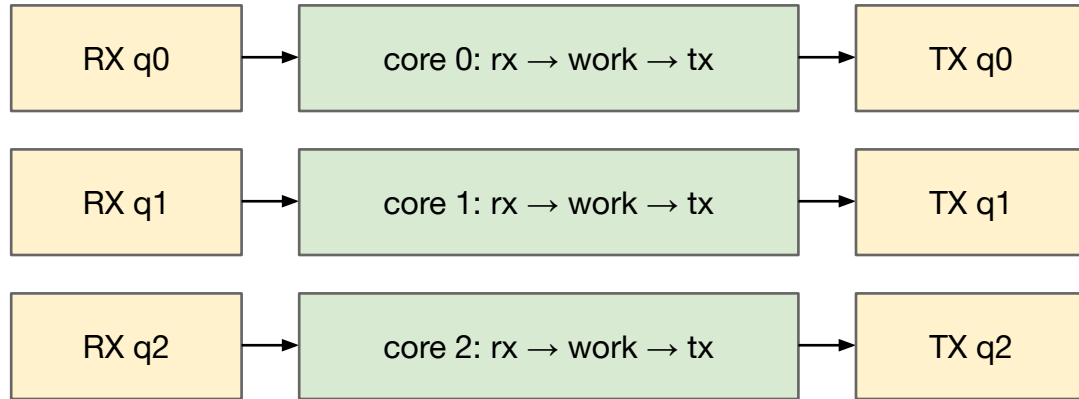
- No locks: each core owns its own queue
- One register write per burst, not per packet
- Prefetching hides memory latency

Vector PMDs

- SSE / AVX2 / AVX-512 / NEON paths handle 4 to 8 descriptors at a time
- Trade a few features for higher packet rate

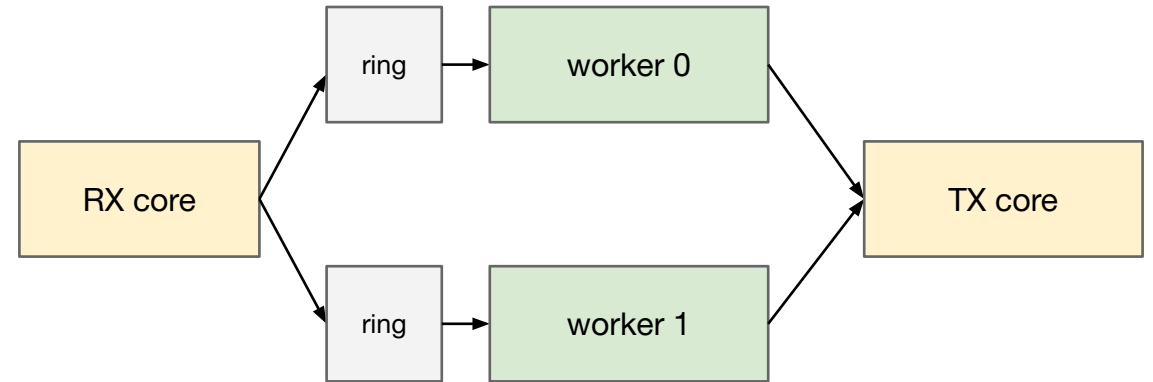
Execution Models: Run-to-Completion vs Pipeline

Run-to-completion



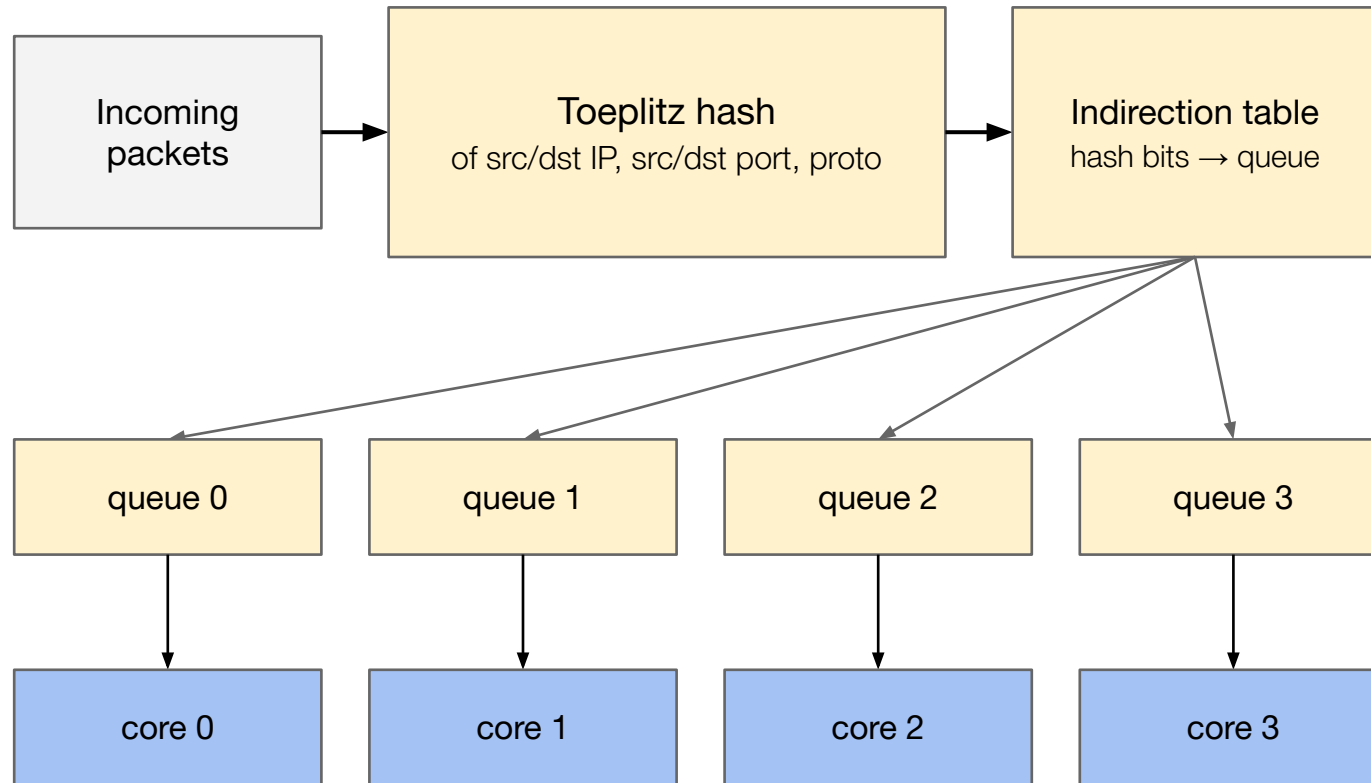
- Best cache locality: a packet stays on one core
- No inter-core rings; scales with RSS queues
- Hard if work per packet varies a lot (head-of-line blocking)

Pipeline



- Stages can use different core counts; easy load balancing
- Fits heavy stages (crypto, DPI) and ordering needs
- Cost: rings between cores, cache misses at each hand-off

Scaling with Multi-queue NICs: RSS and Flow Rules



- Same flow → same queue → same core: per-core state, no locks
- **Symmetric RSS key:** both directions of a connection land on the same core
- **rte_flow:** program match/action rules in the NIC (steer to queue, drop, mark, count)
- Limits: hash imbalance with elephant flows; table size; varies per NIC

Each core polls only its own queue and keeps its own flow state

Hardware Offloads DPDK Can Use

Checksum

Verify IP/TCP/UDP checksums on RX, compute on TX

TSO / LSO

Hand a large TCP buffer; NIC cuts it into segments

LRO

NIC merges consecutive TCP segments on RX

VLAN

Strip tag on RX, insert on TX

Classification

RSS, rte_flow rules, flow marks

Timestamps

Precise RX/TX times, PTP support

Inline crypto

IPsec / MACsec done by the NIC

Tunnels

VXLAN / GENEVE inner checksum and RSS

How: enable at configure time (rxmode.offloads / txmode.offloads), then request or read per packet through mbuf->ol_flags. Check what the NIC supports with rte_eth_dev_info_get().

DPDK: Strengths and Costs

Strengths

- Line-rate small-packet processing: 10+ Mpps per core for simple forwarding
- Wide driver support: Intel, NVIDIA/Mellanox, Broadcom, virtio, cloud NICs
- Rich libraries: LPM, hash, ACL, QoS, crypto, eventdev
- Big ecosystem: OVS-DPDK, VPP, Pktgen, telco 5G user plane; SPDK reuses the same ideas for storage

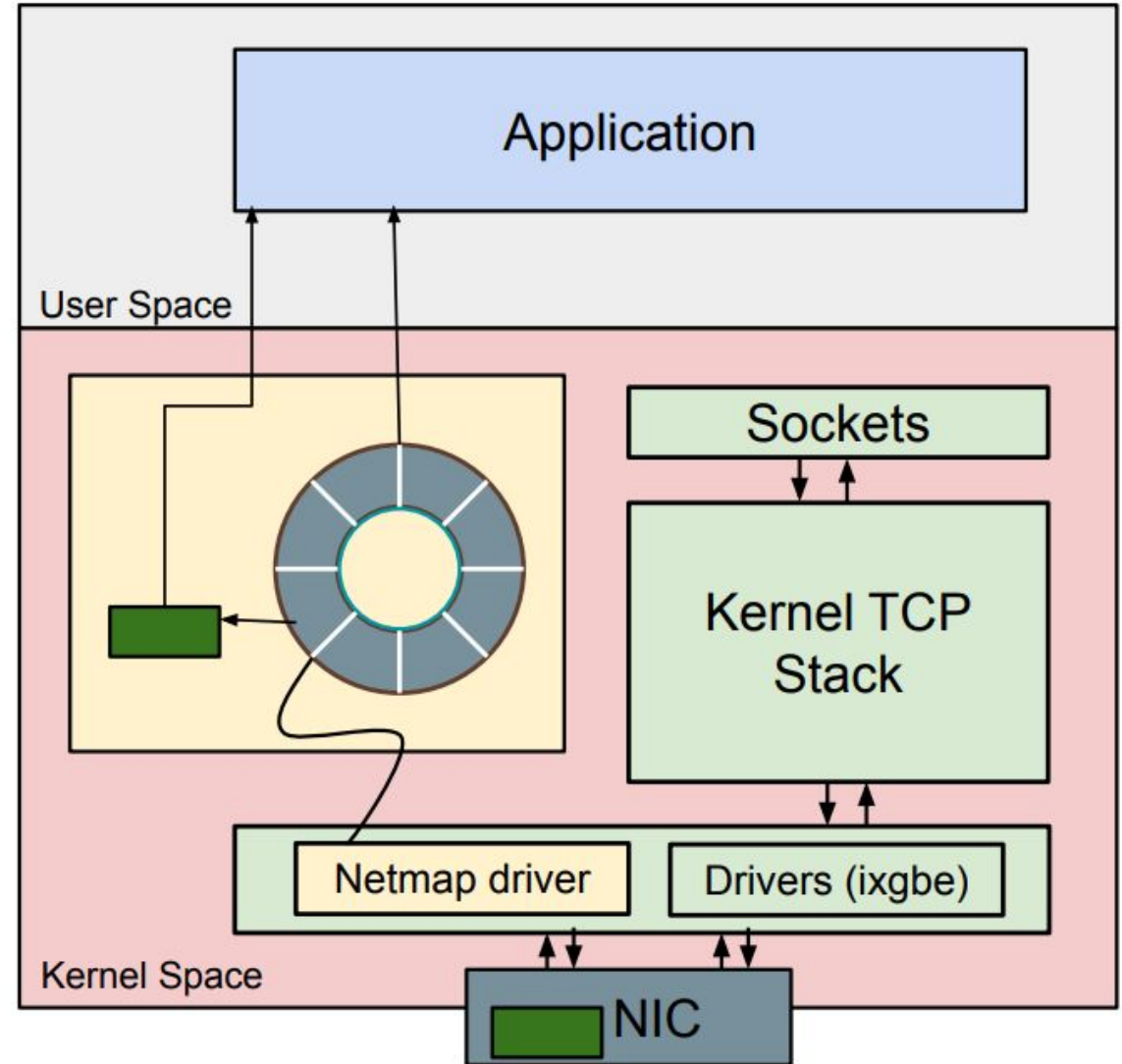
Costs

- Burns 100% of its cores, even with no traffic
- NIC leaves the kernel: no ip, tcpdump or iptables on that port
- Gives raw L2 frames only: **no TCP/IP stack**
- Setup effort: hugepages, VFIO, core isolation, NUMA placement
- App now contains a device driver: bugs are harder to debug

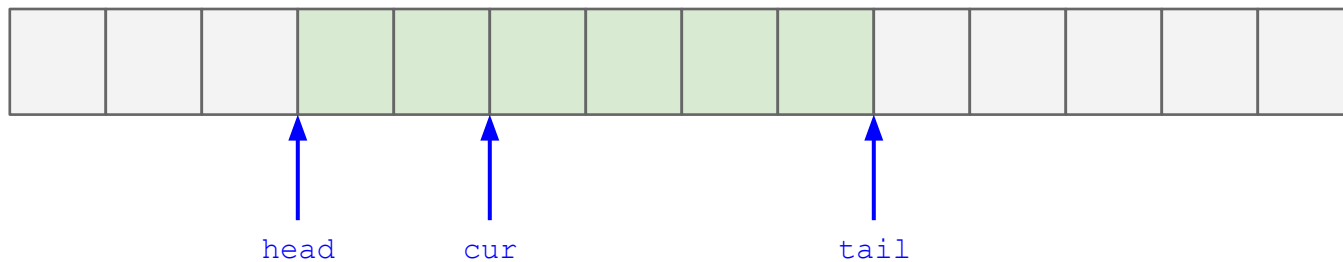
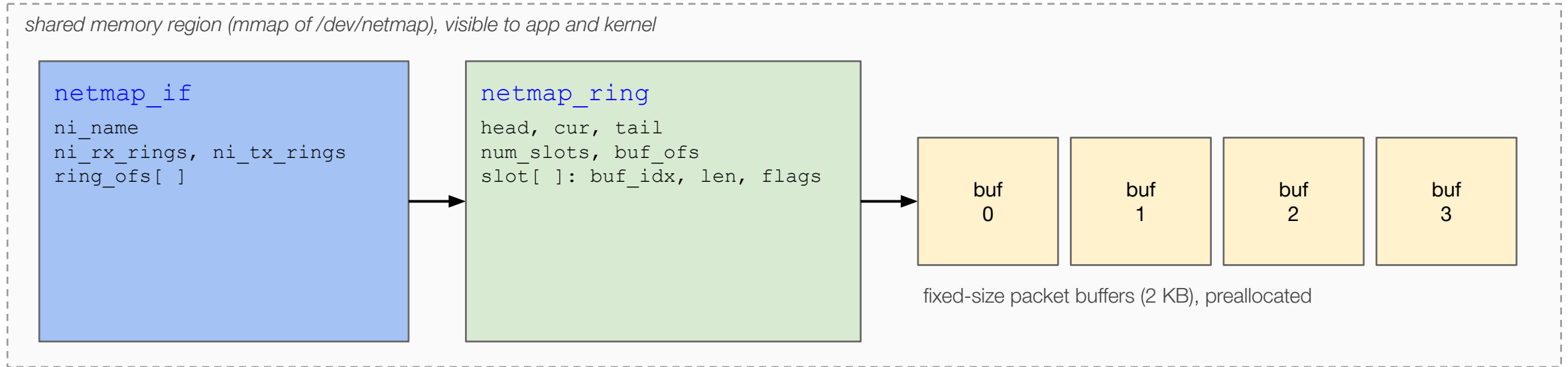
Netmap

- Netmap Rings are memory regions in kernel space shared between application and kernel
- Fast Interface for packet sniffing
- Light-weight packet buffers
- Fewer memory copies
- NIC can work with netmap as well as kernel drivers (transparent mode)

DPDK, netmap manage processing till L2 of network stack



Netmap: Shared Data Structures



RX ring: green slots [head, tail) belong to the app, grey slots to kernel/NIC

- App consumes slots, then advances **head** (and `cur`)
- Kernel advances **tail** on each sync, after the NIC fills slots
- Kernel checks indices before using them

Netmap API: A Minimal Receiver

- `open("/dev/netmap") + ioctl(NIOCREGIF)`: put the port in netmap mode
- `mmap()`: map rings and buffers into the process
- `poll()` / `select()`: wait, and sync the rings in one syscall
- `ioctl(NIOCRXSYNC / NIOCTXSYNC)`: sync without blocking
- **Zero-copy forwarding**: swap `buf_idx` between an RX slot and a TX slot, set `NS_BUF_CHANGED`
- `nm_open()` is the classic helper; newer `libnetmap` offers `nmport_open()`

```
#define NETMAP_WITH_LIBS
#include <net/netmap_user.h>

struct nm_desc *d =
    nm_open("netmap:eth0", NULL, 0, NULL);
struct pollfd pfd = { .fd = d->fd,
                      .events = POLLIN };

for (;;) {
    poll(&pfd, 1, -1); /* 1 syscall, many pkts */
    struct netmap_ring *ring =
        NETMAP_RXRING(d->nifp, 0);
    while (!nm_ring_empty(ring)) {
        uint32_t i = ring->cur;
        char *buf = NETMAP_BUF(ring,
                                ring->slot[i].buf_idx);
        consume(buf, ring->slot[i].len);
        ring->head = ring->cur =
            nm_ring_next(ring, i);
    }
}
```

Why Netmap Is Fast, and Still Safe

Amortised syscalls

One `poll()` or `ioctl()` moves a whole batch of packets between app and NIC

Simple, preallocated buffers

Fixed 2 KB buffers and tiny slot metadata; no `sk_buff`, no allocation per packet

No copies

NIC DMA's into the shared buffers; the app reads them in place

Protection kept

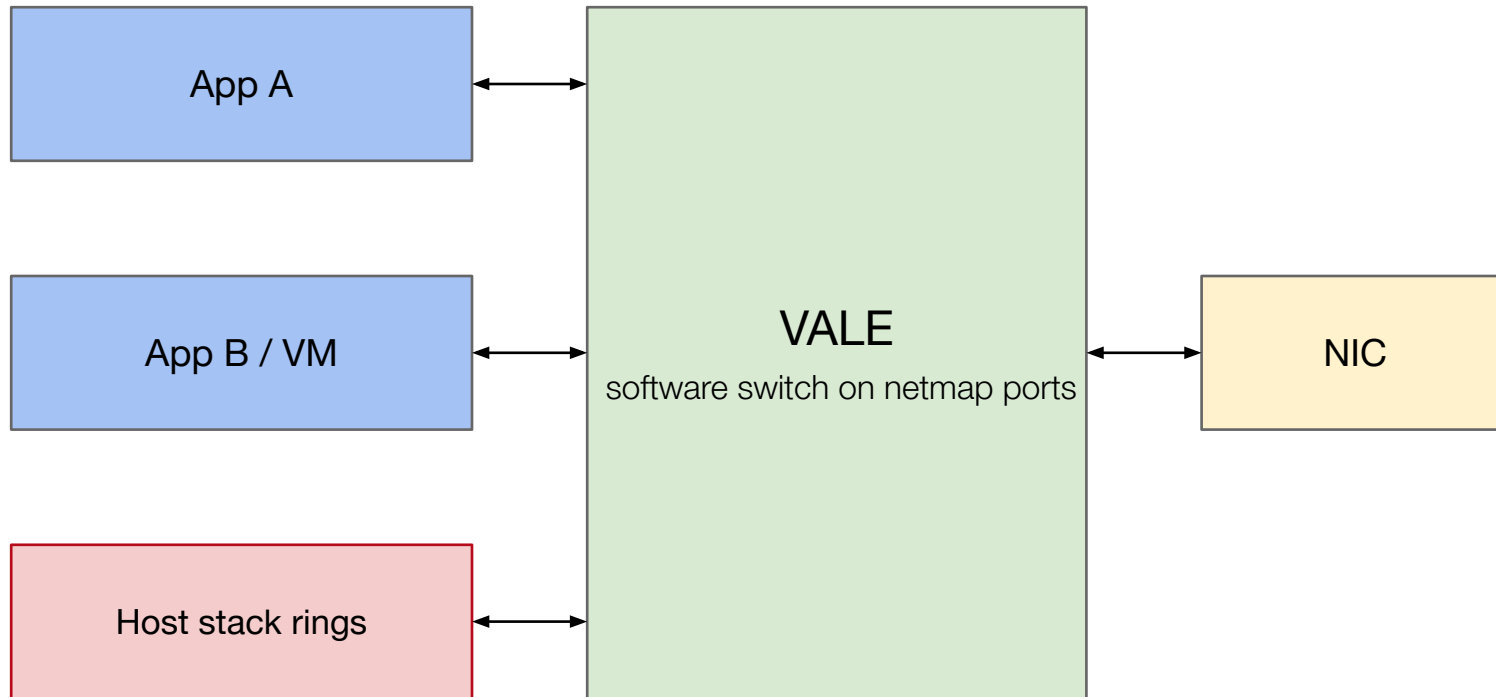
App never touches NIC registers or DMA setup; kernel validates every index on sync, so a buggy app cannot crash the system

14.88 Mpps

10 GbE line rate with 64 B packets on a single core clocked below 1 GHz (Rizzo, ATC 2012)

The NIC still uses the kernel driver (modified). Interrupts are used when the app sleeps in `poll()`.

Netmap Beyond a Single NIC



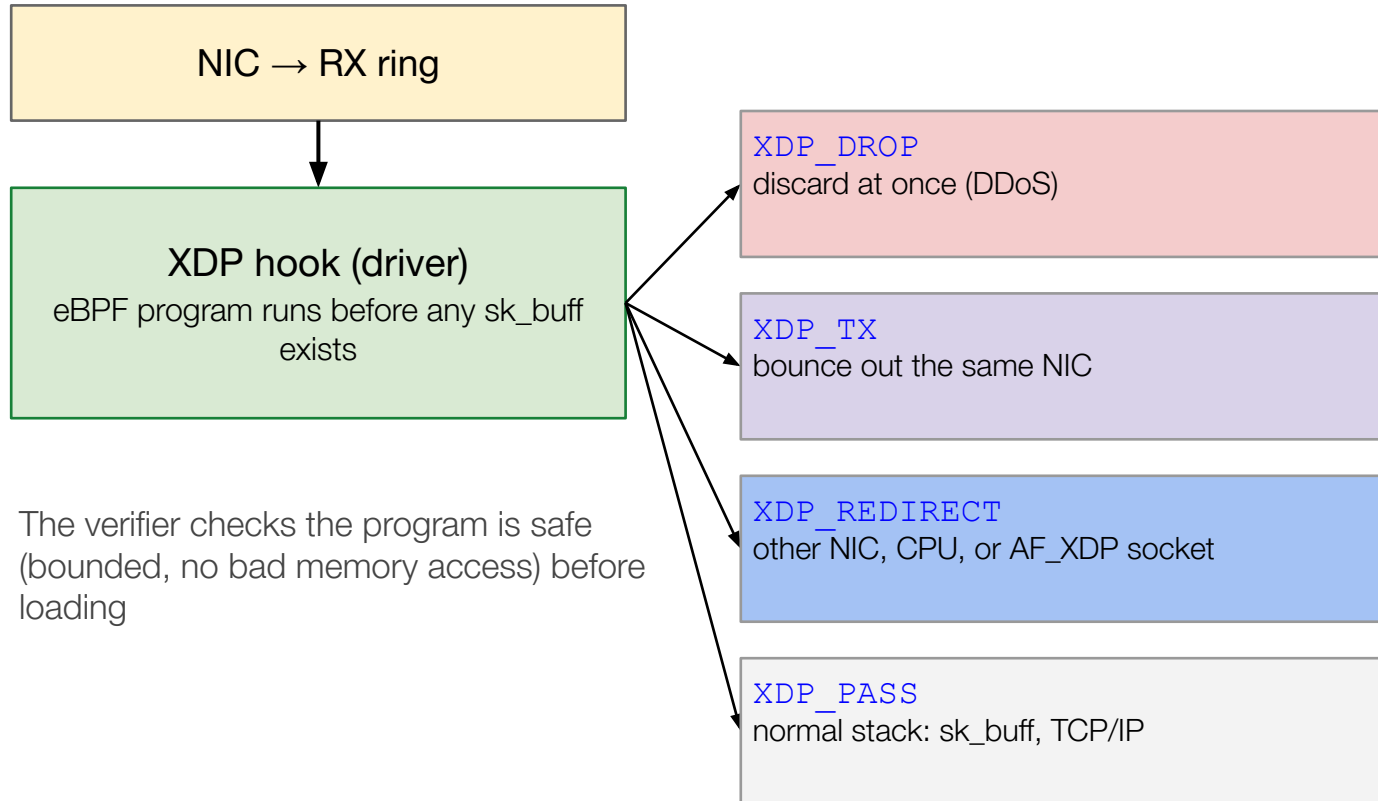
- **Host rings:** pass packets to and from the normal kernel stack (transparent mode)
- **VALE:** fast software L2 switch between apps and VMs
- **Pipes:** shared memory channels between processes
- **ptnet / VM backends:** used by bhyve and QEMU
- Part of FreeBSD; loadable module on Linux; tools such as pkt-gen

Same ring API everywhere: a program does not care whether its port is a NIC, a switch port or a pipe.

DPDK vs Netmap

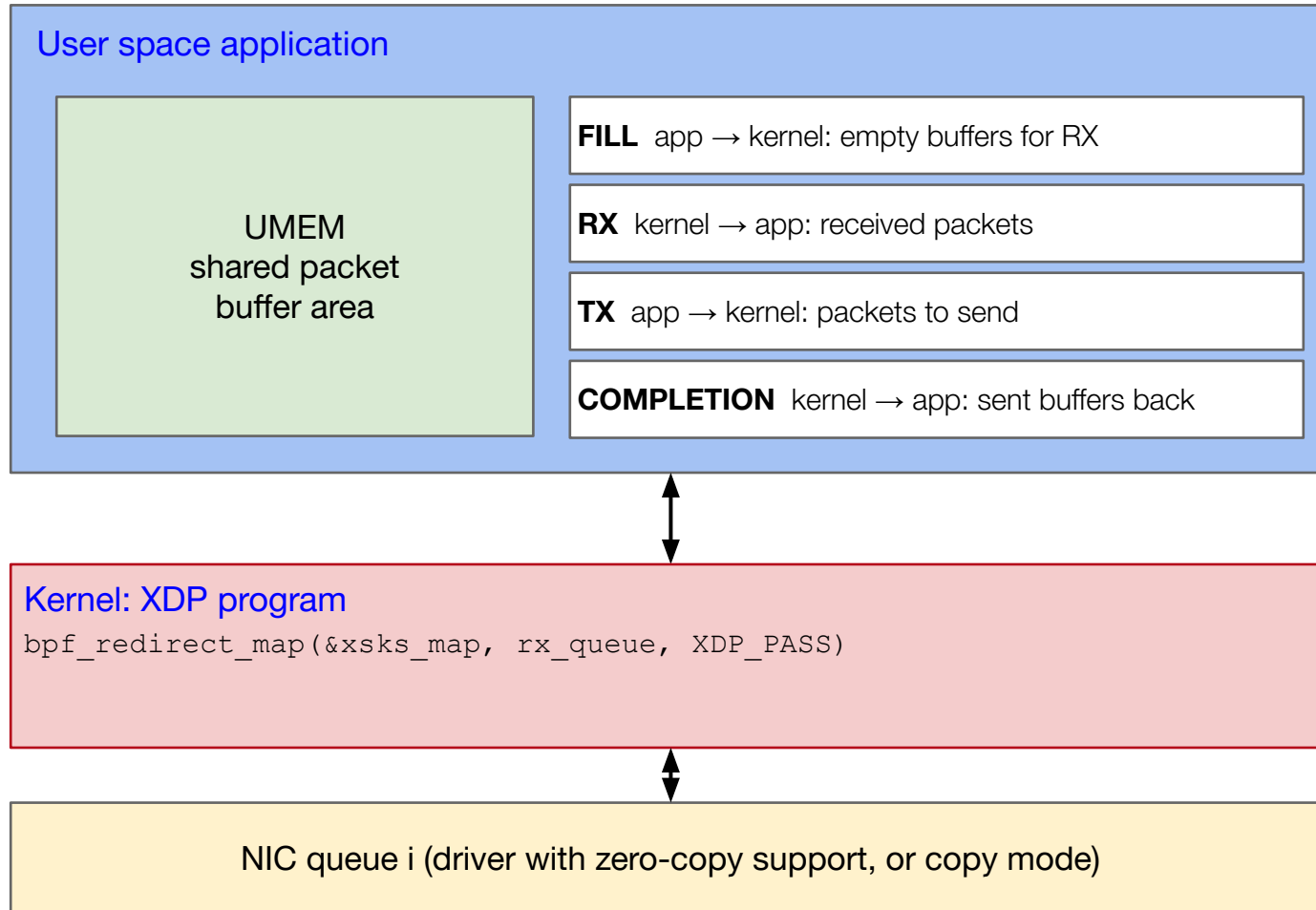
Aspect	DPDK	Netmap
Where the driver runs	User space (PMD library)	Kernel (modified native driver)
Who owns the NIC	The app; kernel loses the port	Kernel; can switch modes any time
Notification	Busy polling (optional Rx interrupts)	poll()/select(), or busy ioctl sync
Syscalls on data path	None	One per batch
Buffers	rte_mbuf in hugepage mempools	Fixed 2 KB buffers in shared region
Protection	App drives the device; IOMMU limits DMA	Kernel validates; app never touches registers
Path to kernel stack	Extra pieces (TAP, virtio-user)	Built-in host rings
Ecosystem	Very large: libraries, drivers, OVS, VPP	Smaller; FreeBSD, VALE, research tools
Typical use	NFV, telco, load balancers, appliances	Capture, generators, software switches

In-kernel Fast Path: XDP



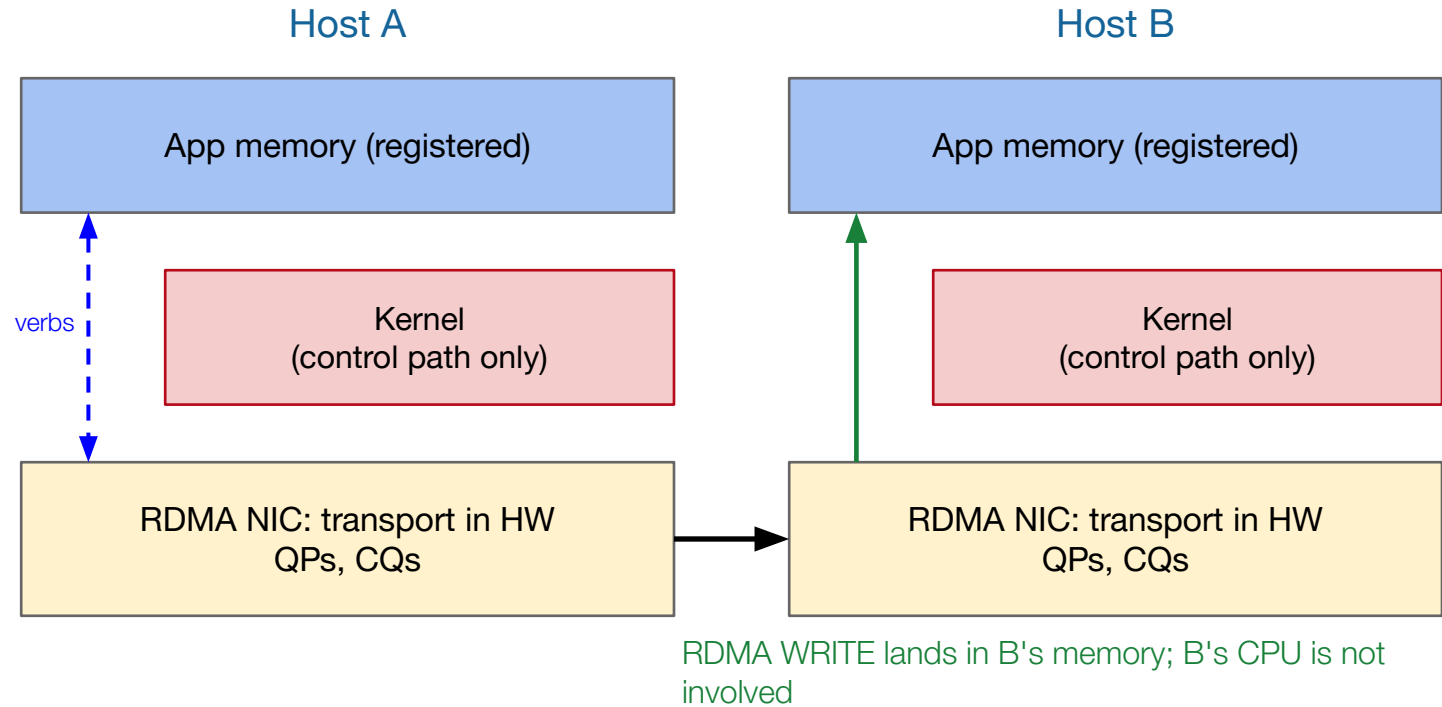
- Runs **inside the kernel**, so the NIC stays shared with normal traffic and tools
- No NIC takeover, no hugepage setup
- About 24 Mpps per core for dropping packets (Høiland-Jørgensen et al., CoNEXT 2018)
- In production: Facebook Katran load balancer, Cloudflare DDoS filtering
- Limits: small, restricted programs; no full protocol stack

AF_XDP: Raw Packets to User Space via XDP



- One AF_XDP socket per NIC queue
- Four single-producer / single-consumer rings, like netmap and DPDK
- **Zero-copy mode:** NIC DMAs straight into UMEM
- Other traffic still goes to the kernel stack (XDP_PASS)
- Standard Linux, no extra driver; DPDK even has an af_xdp PMD
- Speed between kernel sockets and full DPDK

Another Kind of Bypass: RDMA

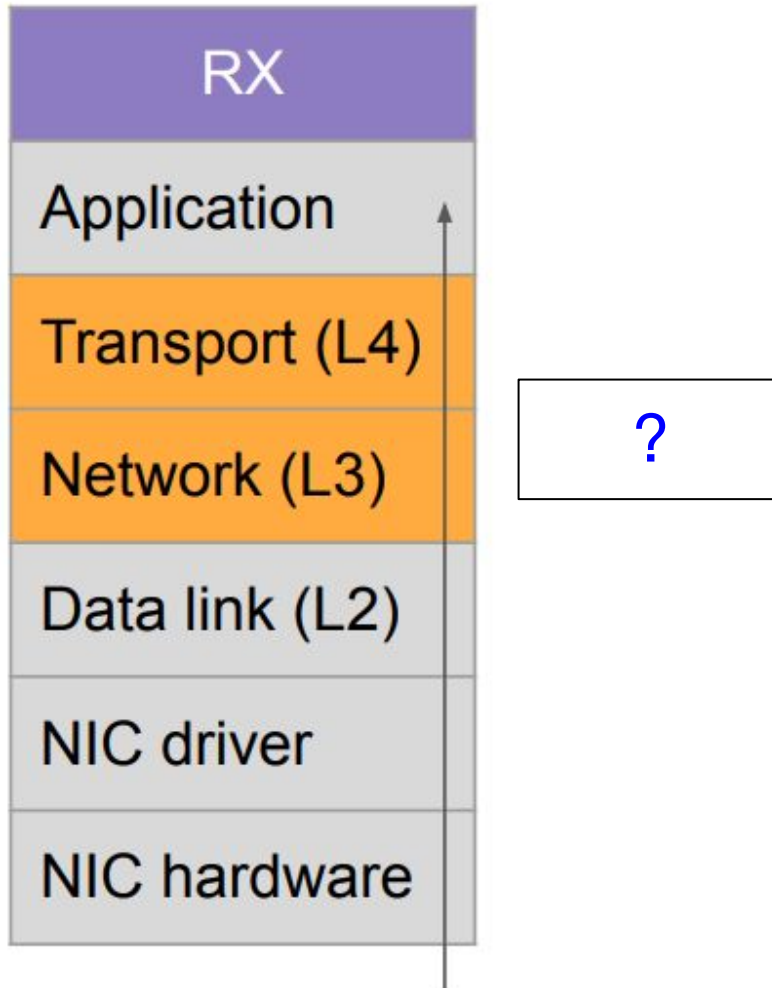


- App posts work requests on **queue pairs** straight to the NIC (no syscall)
- NIC runs the transport: reliability, segmentation, ordering
- **One-sided** READ/WRITE and **two-sided** SEND/RECV
- App polls **completion queues**
- Memory must be registered (pinned and mapped for the NIC)

Networks: InfiniBand, RoCEv2 (RDMA over UDP/IP on Ethernet), iWARP (over TCP)

Used in HPC (MPI), storage (NVMe-oF), distributed ML (NCCL, GPUDirect), key-value stores

L3/L4 Processing



Why Not Just Keep the Kernel TCP Stack?

- DPDK and netmap give **raw frames**; most applications speak TCP
- Many workloads are dominated by **short connections** and small messages (web, RPC, mobile apps)
- For these, the kernel stack suffers from:
 - shared listen queue and socket structures
 - no connection locality across cores
 - file descriptor and VFS overhead
 - one syscall per small read or write
- In-kernel attempts: Affinity-Accept (EuroSys 2012), MegaPipe (OSDI 2012), SO_REUSEPORT. They fix parts only

Observation (mTCP, NSDI 2014)

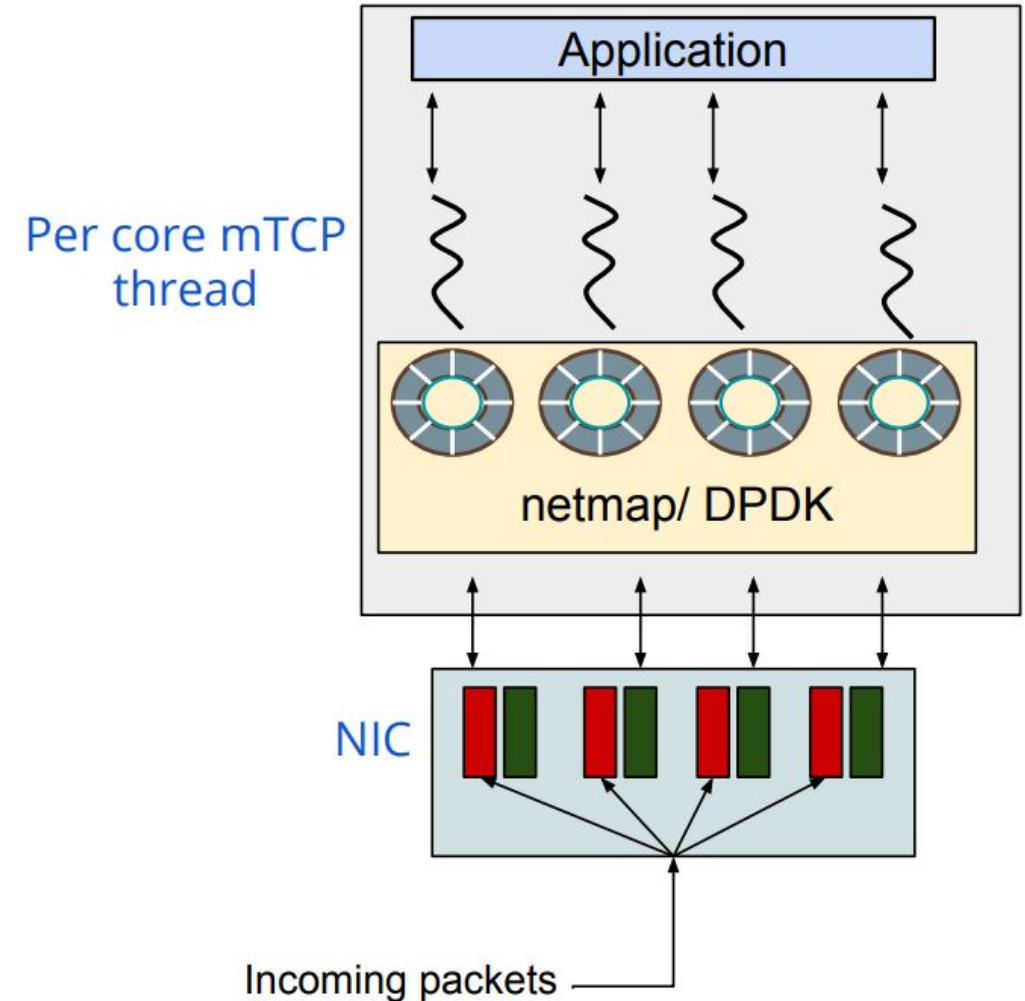
For short flows, most CPU time goes to the kernel (TCP/IP, packet I/O, syscalls), and only a small share is left for the application itself.

Idea

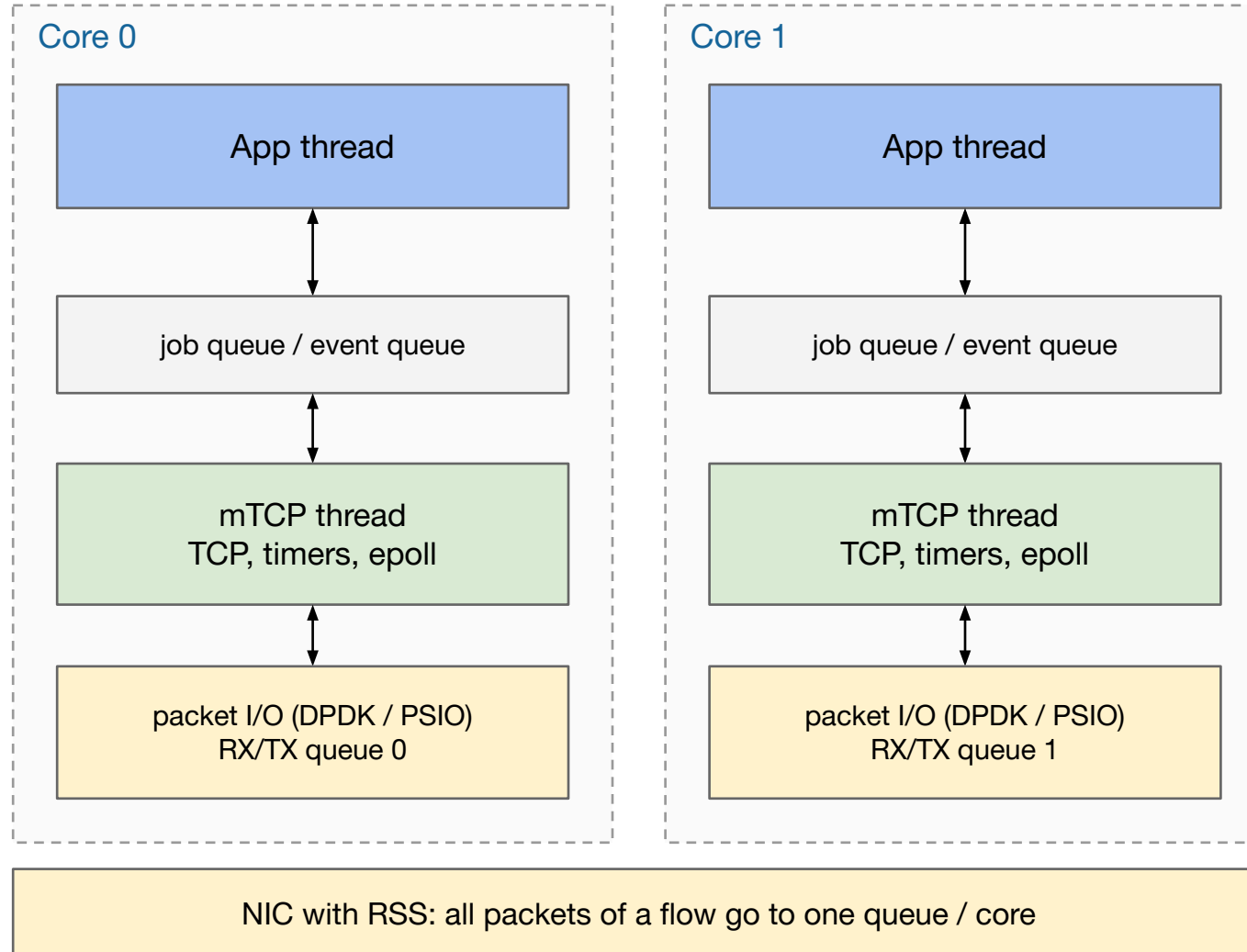
Move TCP into user space, on top of a kernel-bypass packet I/O engine, and design every structure per core. This is **mTCP**.

mTCP: Userspace network stack

- User-space TCP/IP stack built over kernel bypass packet I/O engines(e.g. DPDK)
- Designed for multicore scalable application
- Per core TCP data structures
 - E.g. accept queue, socket list
 - Lock free
 - Connection locality
- Leverages multiqueue support of NIC
- No shared data structures

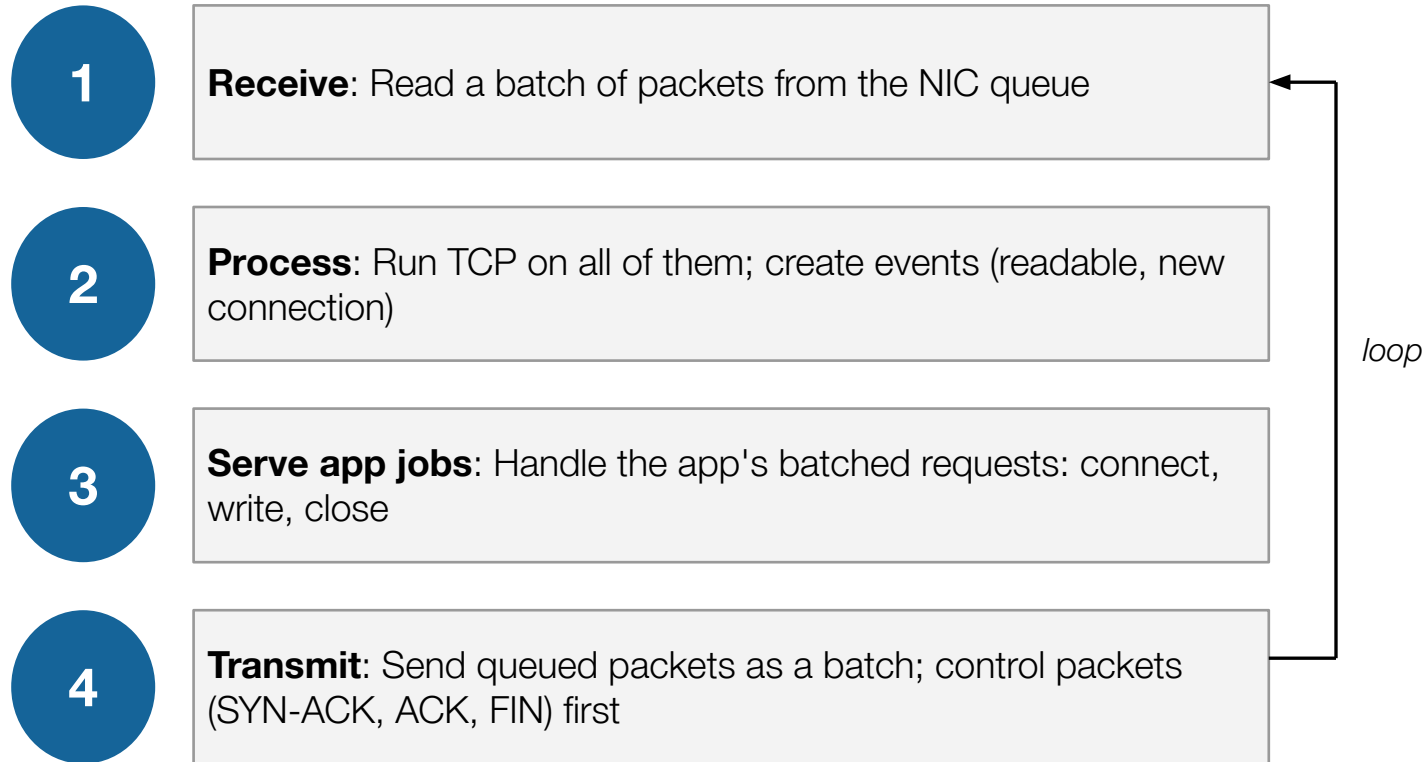


mTCP Architecture: Per-core Threads



- Each core runs one **app thread** and one **mTCP thread**, both pinned
- They talk through per-core queues, not syscalls
- Per-core accept queue, socket table, flow table, memory pools: **no sharing, no locks**
- RSS keeps each connection on one core (connection locality)
- Own epoll-like event system for the app

mTCP: Batching and Event Handling



- Batching everywhere: packet I/O, TCP processing, app requests, events
- Fewer cache misses, fewer thread switches per packet
- Control packets get priority so connection setup stays fast under load
- Per-core timer lists for retransmission and TIME_WAIT

Switching between app and mTCP threads still costs a context switch, but it is amortised over a whole batch of events and packets.

mTCP API: Porting an Application

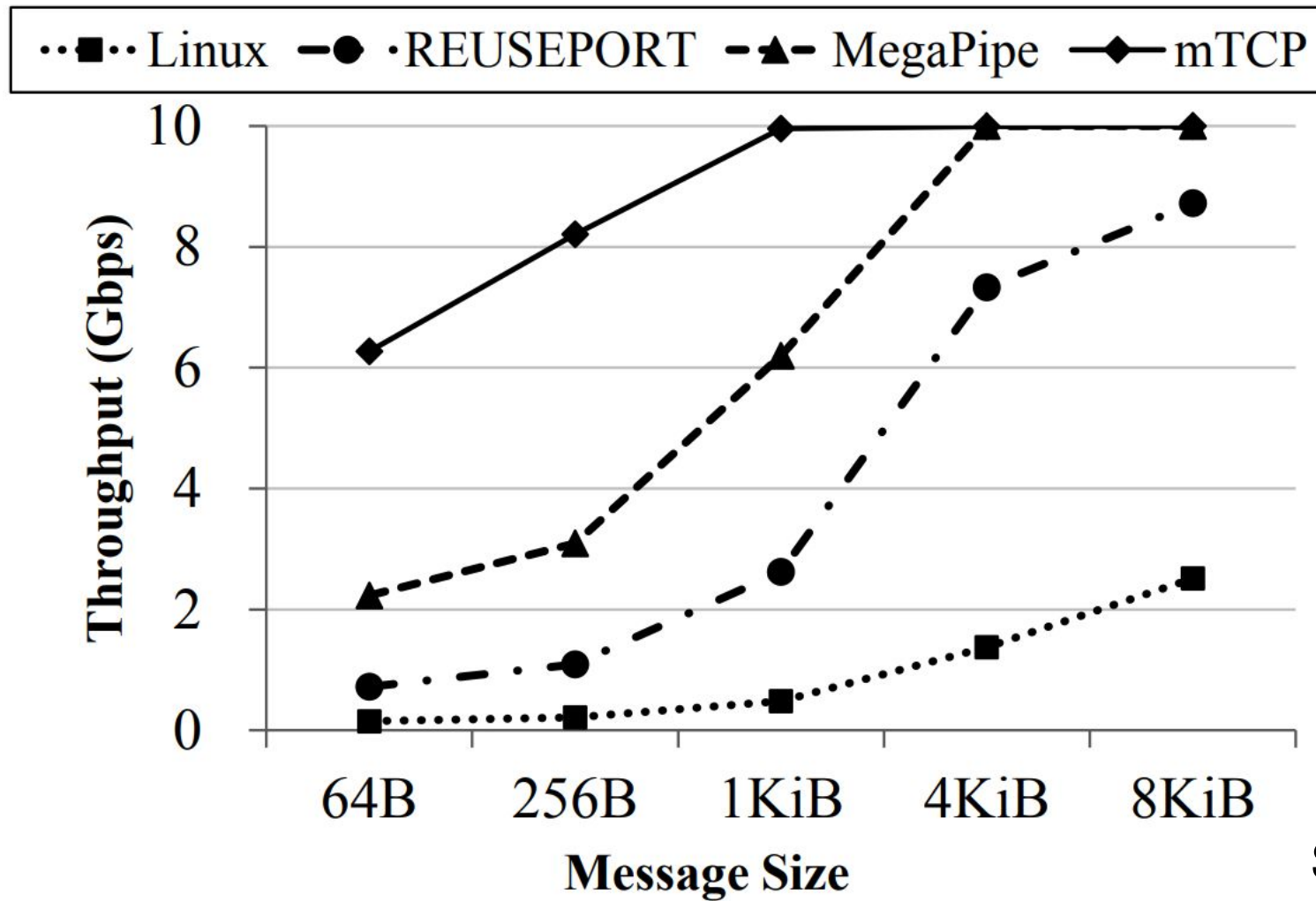
```
mtcp_init("mtcp.conf");
mtcp_core_affinitize(core);
mctx_t mctx = mtcp_create_context(core);

int ep = mtcp_epoll_create(mctx, MAX_EV);
int ls = mtcp_socket(mctx, AF_INET,
                    SOCK_STREAM, 0);
mtcp_setsock_nonblock(mctx, ls);
mtcp_bind(mctx, ls, (struct sockaddr *)&sa,
          sizeof(sa));
mtcp_listen(mctx, ls, 4096);
/* add ls to ep with MTCP_EPOLLIN */

for (;;) {
    int n = mtcp_epoll_wait(mctx, ep, ev,
                           MAX_EV, -1);
    for (int i = 0; i < n; i++) {
        int s = ev[i].data.sockid;
        if (s == ls) mtcp_accept(mctx, ls, NULL, NULL);
        else mtcp_read(mctx, s, buf, sizeof(buf));
    }
}
```

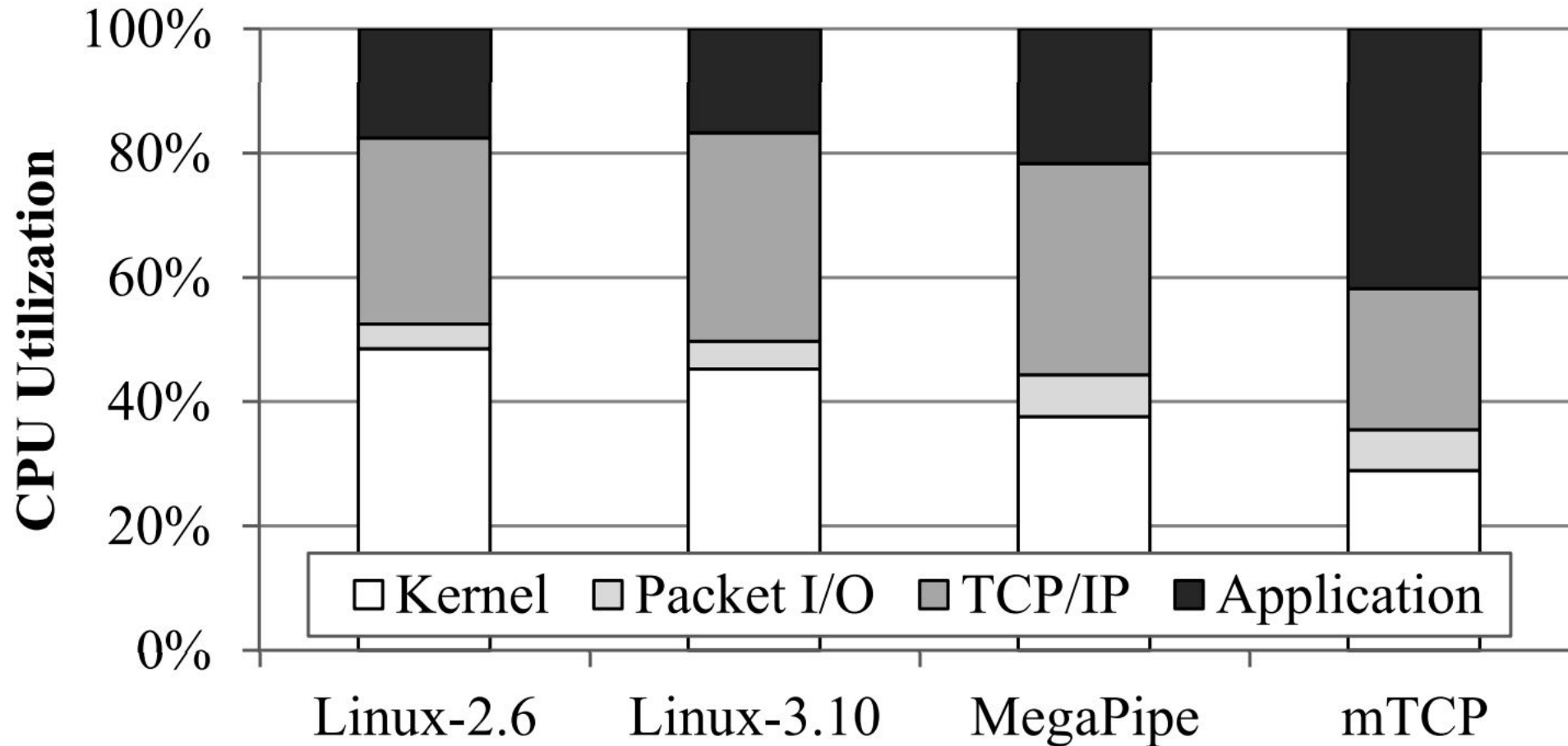
- Looks like BSD sockets + epoll, with an extra **mctx** (per-core context)
- One context and one event loop per core
- Authors ported lighttpd, ApacheBench and SSLShader with modest code changes
- Limits: not a drop-in POSIX replacement; no sharing sockets across processes; the NIC port belongs to this application

mTCP Performance



Source: [mTCP paper](#)

mTCP Performance (Contd.)



Source: [mTCP paper](#)

Landscape of Bypass and User-space Stacks

System	Origin	Key idea
mTCP	NSDI 2014	Per-core user-space TCP on DPDK/PSIO, batching
IX	OSDI 2014	Protected dataplane OS (virtualisation), run-to-completion, adaptive batching
Arrakis	OSDI 2014	Kernel as control plane; apps use SR-IOV virtual NICs directly
Seastar	ScyllaDB	Shared-nothing C++ framework with its own TCP stack on DPDK
F-Stack	Tencent	FreeBSD TCP/IP stack moved to user space on DPDK
TAS	EuroSys 2019	TCP fast path as a separate service on dedicated cores
Snap	SOSP 2019 (Google)	Microkernel-style user-space networking in production
eRPC	NSDI 2019	Fast datacenter RPC over plain Ethernet with bypass
Demikernel	SOSP 2021	Library OS with one API over DPDK, RDMA and SPDK
VPP (fd.io)	Cisco / LF	Vector packet processing: batches flow through a graph of nodes

Outline

- Linux Network Stack
 - Receive path, NAPI, transmit path
- Need for Kernel Bypass
 - Per-packet time budget and kernel overheads
- Kernel Bypass Techniques
 - Core ideas: direct NIC access, polling, batching
 - User-space packet processing: DPDK and Netmap
 - In-kernel fast path: XDP and AF_XDP; RDMA
 - User-space network stack: mTCP
- **Challenges and when to use kernel bypass**

Challenges and When to Use Kernel Bypass

Challenges of Kernel Bypass

Lost kernel features

No iptables/nftables, tc, routing tables, ss or tcpdump on bypassed ports. Rebuild what you need

Security and isolation

App holds device access; IOMMU limits DMA. Sharing a NIC among tenants needs SR-IOV or flow rules

CPU and energy

Polling cores run at 100% even when idle. Hard to share cores; poor fit for bursty or low traffic

Getting TCP right

Congestion control, timers and corner cases are hard. User stacks lag kernel features

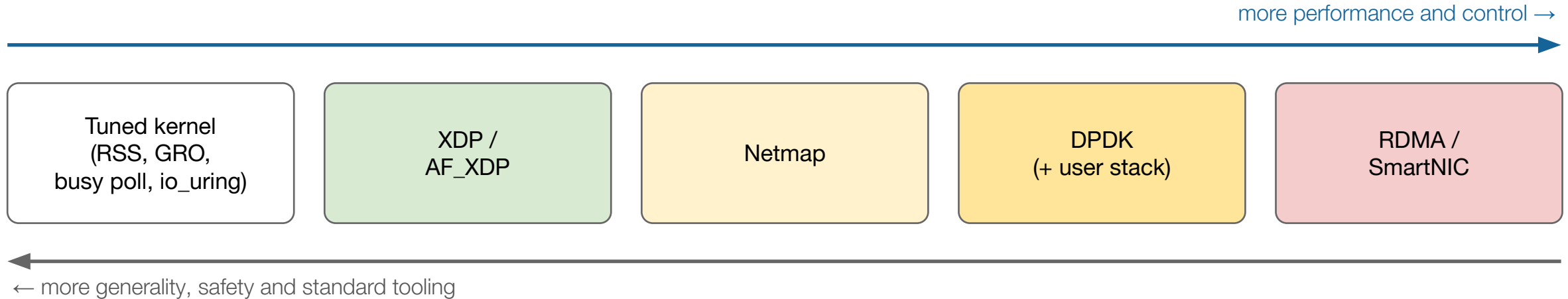
Operations and debugging

No standard tools; need own counters and tracing. A restart drops every connection

Portability

NIC-specific drivers and offloads, API churn across releases, apps must be changed

When Should You Use Kernel Bypass?



Good fit

- Network functions: routers, firewalls, 5G user plane
- L4 load balancers, DDoS scrubbing, packet capture / IDS
- Traffic generators, trading systems, high-rate RPC and key-value stores

Poor fit

- General-purpose servers with modest traffic
- Many applications sharing one NIC
- Teams that depend on standard Linux tools and kernel TCP features

Summary

- The Linux stack is general and safe, but per-packet costs (interrupts, sk_buff, copies, syscalls, shared state) keep it far below line rate for small packets
- At 100 Gbps there are only ~6.7 ns per minimum-size packet
- Kernel bypass: map the NIC into user space, poll, batch, preallocate, pin cores, share nothing
- **DPDK**: user-space drivers and libraries; top speed, but the NIC leaves the kernel
- **Netmap**: kernel keeps the driver; shared rings and batched syscalls keep protection
- **XDP / AF_XDP**: eBPF fast path inside the kernel; **RDMA** moves the transport into the NIC
- **mTCP** and friends: user-space TCP designed per core for connection-heavy workloads
- Choose by workload: performance and control vs generality and tooling

References

- [DPDK Programmer's Guide](#)
- [DPDK Getting Started Guide for Linux](#)
- [L. Rizzo, netmap: a novel framework for fast packet I/O, USENIX ATC 2012](#)
- [netmap implementation \(GitHub\)](#)
- [E. Jeong et al., mTCP: a Highly Scalable User-level TCP Stack for Multicore Systems, NSDI 2014](#)
- [mTCP implementation \(GitHub\)](#)
- [T. Høiland-Jørgensen et al., The eXpress Data Path, CoNEXT 2018](#)
- [Linux kernel documentation: AF_XDP](#)
- J. Mogul, K. K. Ramakrishnan, Eliminating Receive Livelock in an Interrupt-driven Kernel, ACM TOCS 1997
- A. Belay et al., IX, OSDI 2014; S. Peter et al., Arrakis, OSDI 2014
- [Cloudflare blog on kernel bypass](#)
- Linux network stack slides: Mythili Vutukuru, IIT Bombay